Below is a **master "Java Terminology Dictionary"** covering **all major Java versions (Java 1.0 → Java 21+)**, including the **most important terminologies used in real-world software development**, with **use cases + small examples**, in a **table format**.

✅ Note: "List of ALL terminology" in Java is practically infinite (because Java has libraries + frameworks + JVM internals).
So I'm giving you the **complete enterprise-level terminology set** that is used in **real-time applications**, interviews, and production systems.

✅ Java Terminology (All Versions) + Real-Time Use Cases + Examples (MASTER TABLE)

Terminology	Meaning (Detailed)	Introduced / Famous In	Real-Time Use Case (Software)	Example
JDK	Java Development Kit (compiler + tools + JRE)	1.0+	Used for building apps, compilation, packaging	javac App.java
JRE	Runtime environment to run Java programs	1.0+	For running deployed apps	java App
JVM	Virtual machine that executes bytecode	1.0+	Backend services run on JVM (Spring Boot)	Runs .class
Bytecode	Intermediate compiled code (.class)	1.0+	Enables portability across OS	javac output
WORA	Write Once Run Anywhere	1.0+	Deploy same jar on Linux/Windows servers	.jar
Class	Blueprint of objects	1.0+	Model class in enterprise (User, Order)	class User{}
Object	Instance of class	1.0+	Real entity in runtime	new User()
Constructor	Initializes object	1.0+	Creating DTO/entity in applications	User(){}

Method	Function inside class	1.0+	Service layer method (createOrder)	save()
Variable	Stores data	1.0+	All programs	int age=10;
Primitive Types	int, double, char, boolean etc.	1.0+	Performance critical logic	int x=5;
Reference Types	Objects like String, List	1.0+	Entity objects, collections	String s
Wrapper Classes	Object forms of primitives	1.0+	Required for Collections / generics	Integer i=1
String Pool	Memory optimization for Strings	1.0+	Performance improvements in large apps	"abc" stored
Immutable	Object cannot change	1.0+	Thread safety, caching	String, LocalDate
Mutable	Object can change	1.0+	Builders, buffer operations	StringBuilder
OOP	Object oriented programming	1.0+	All enterprise systems are OOP based	Encapsulation
Encapsulation	Hide internal data with getters/setters	1.0+	Domain models	private fields
Inheritance	Parent-child relationship	1.0+	BaseEntity, BaseService	extends
Polymorphism	Same method works differently	1.0+	Payment = UPI/Card/NetBanking	override
Abstraction	Hide implementation details	1.0+	Interfaces in microservices	interface
Interface	Contract of behavior	1.0+	PaymentGateway, Repository	implements
Abstract Class	Partial abstraction	1.0+	Base workflow, template logic	abstract

Overloading	Same method name with different parameters	1.0+	Utility methods	add(int), add(double)
Overriding	Subclass redefines method	1.0+	Implementation in services	@Override
Static	Belongs to class not object	1.0+	Utility methods/constants	static final
Final	Prevent change/inheritance/override	1.0+	Constants, security	final int
this keyword	Refers current object	1.0+	Constructor chaining	this.name=name
super keyword	Refers parent	1.0+	Parent method access	super()
Access Modifiers	public/private/protected/default	1.0+	Encapsulation + API exposure	private
Package	Namespace organization	1.0+	Large microservices grouping	com.app.user
Import	Imports classes	1.0+	Multi-module code	import java.util.*;
Exception	Runtime abnormal condition	1.0+	Error handling in APIs	try-catch
Checked Exception	Compile-time mandatory handling	1.0+	IO, DB, File operations	IOException
Unchecked Exception	Runtime exception	1.0+	Validation issues	NullPointer Exception
try-catch-finally	Exception handling flow	1.0+	API + DB safe close	finally{}
throw	Manually throw exception	1.0+	Business rules failure	throw new...
throws	Declare exception	1.0+	Library/service method signature	throws IOException

Garbage Collection	Automatic memory cleanup	1.0+	Prevent memory leak in server	GC runs
Heap	Memory area for objects	1.0+	JVM apps store entities on heap	new uses heap
Stack	Thread local memory	1.0+	Method calls, local vars	recursion uses stack
ClassLoader	Loads classes dynamically	1.0+	Spring Boot loads beans	AppClassLoader
JIT	Just-In-Time compilation	1.0+	Runtime optimization	HotSpot
HotSpot	JVM optimization engine	1.3+	Enterprise performance	C2 compiler
Reflection	Inspect classes at runtime	1.1+	Frameworks like Spring/Hibernate	Class.forName
Annotation	Metadata for runtime/compile-time	5+	Spring @Autowired, JPA @Entity	@Override
Generics	Type safety in collections	5	Avoid ClassCastException	List<String>
Collections Framework	List, Set, Map etc.	1.2	API responses, caching, DB data	HashMap
HashMap	Key-value store	1.2	Fast lookup (cache)	map.put()
Concurrent HashMap	Thread-safe map	5	Multi-thread caching	new ConcurrentHashMap()
Stream API	Functional operations on data	8	Filtering/sorting API responses	.stream().filter()
Lambda	Anonymous function	8	Cleaner async/event code	x -> x+1

Functional Interface	Single abstract method interface	8	Used in lambdas	Runnable
Optional	Avoid null handling	8	Prevent NPE in service layer	Optional.of()
Default Method	Method body in interface	8	Backward compatibility in APIs	default void
Method Reference	Cleaner lambda reference	8	Event handlers, streams	System.out::println
Completable Future	Async computation	8	Parallel API calls in microservices	supplyAsync()
ForkJoinPool	Parallelism framework	7	CPU parallel tasks	Parallel streams
NIO	Non-blocking IO	1.4+	High performance file/network apps	java.nio
NIO.2	Improved file APIs	7	File watching, paths	Files.readAllBytes()
Modules	Modular Java runtime	9	Reduce runtime size, strong encapsulation	module-info.java
JShell	Java REPL shell	9	Quick testing/snippets	Jshell
Var	Local variable type inference	10	Cleaner code in enterprise	var list = ...
Records	Immutable data carrier	16	DTO for REST APIs	record User (int id)
Sealed Classes	Restrict inheritance	17	Domain safety in apps	sealed class
Pattern Matching	Smarter instanceof/switch	16-21	Cleaner validation/dispatch	if(obj instanceof X x)
Virtual Threads	Lightweight threads	21	Handle massive concurrency	Thread.startVirtualThread()

			(10k requests)	
Structured Concurrency	Safe thread grouping	21	Scoped parallel tasks	StructuredTaskScope
Scoped Values	Better ThreadLocal alternative	21	Request context propagation	scoped context
Text Blocks	Multi-line strings	15	JSON/XML writing	"""
Switch Expressions	Return values from switch	14	Cleaner mappings	switch(x){...}
GC Types	Serial, Parallel, G1, ZGC, Shenandoah	multiple	Performance tuning backend	-XX:+UseG1GC
JNI	Java Native Interface	1.0+	Integrate C/C++ libs	native method
JAR	Java Archive packaging	1.0+	Deploy services	java –jar
WAR	Web archive	Java EE era	Deploy to Tomcat	app.war
Maven	Dependency build tool	widely used	Build Spring projects	pom.xml
Gradle	Modern build tool	widely used	Android + modern builds	build.gradle
Unit Testing	Test single units	JUnit	CI/CD pipelines	@Test
Mocking	Fake dependencies	Mockito	Service testing	when().thenReturn()
JPA	Java persistence API	Java EE	ORM mapping	@Entity
Hibernate	ORM implementation	widely used	Database save/find	session.save()
JDBC	DB connectivity API	1.1	SQL integration	PreparedStatement
Connection Pool	Reuse DB connections	enterprise	Performance optimization	HikariCP

Serialization	Convert object → bytes	1.0+	Cache, sessions	implements Serializable
Deserialization	bytes → object	1.0+	Read saved objects	ObjectInputStream
Lock	Thread synchronization control	5+	Prevent race conditions	ReentrantLock
synchronized	Monitor lock keyword	1.0+	Critical section protection	synchronized(obj)
volatile	Visibility guarantee	1.0+	Multi-thread flags	volatile boolean
Deadlock	Threads stuck forever	1.0+	Must avoid in banking systems	lock order
Race Condition	Wrong data due to concurrency	1.0+	Payments & wallets	shared state
Microservices	Split application into services	modern Java	Large scale systems	Spring Cloud
Monolith	Single large application	classic	Small org app	single deploy
REST API	HTTP API architecture	modern	Mobile + web backend	GET/POST
SOAP	XML-based web services	older enterprise	Legacy banking systems	WSDL
JVM Flags	Runtime tuning options	all	Prod tuning	-Xms -Xmx

✅ Real Enterprise Example Snippets (Quick)

✅ 1) Java 8 Stream + Lambda (real-time filtering)

```
List<User> active = users.stream()
    .filter(u -> u.isActive())
    .collect(Collectors.toList());
```

✅ 2) Java 16 Record for DTO

```
public record LoginRequest(String username, String password) {}
```

✅ 3) Java 21 Virtual Threads (high concurrency)

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
   executor.submit(() -> System.out.println("Hello virtual thread"));
}
```

If you want, I can also provide:

✅ **A PDF-ready master table** (1000+ terms)
✅ Version-wise categorized: Java 1.0–1.4, 5, 7, 8, 9, 11, 17, 21
✅ **Separate terminology tables** for:

- JVM Internals terminology
- Java Concurrency terminology
- Java Memory Model terminology
- Java Collections terminology
- Java 8+ Functional terminology
- Spring Boot / Microservices terminology (enterprise)

Just tell me: **Do you want "version-wise tables" or "category-wise tables"?**

I will generate **1000+ Java terms** here **in PARTS**, in a **table format**, with:

- ✅ **Term**
- ✅ **Meaning (detailed but crisp)**
- ✅ **Java Version / Era**
- ✅ **Real-time Use Case**
- ✅ **Mini Example**

□ I'll start with:

✅ PART 1 (Terms 1–120) — Core Java + OOP + JVM Basics

✅ Java 1000+ Terminology MASTER LIST (PART 1: 1–120)

#	Term	Meaning	Version/Era	Real-Time Use Case	Example
1	Java	High-level OOP language running on JVM	1.0+	Backend apps, Android (legacy), enterprise apps	Spring Boot

2	JDK	Dev kit: compiler + tools + libs	1.0+	Building apps & services	javac
3	JRE	Runtime environment to run Java apps	1.0+	Running deployed apps	java App
4	JVM	Virtual machine executing bytecode	1.0+	Run microservices	HotSpot
5	Bytecode	.class intermediate code	1.0+	Cross-platform execution	.class file
6	WORA	Write Once Run Anywhere concept	1.0+	Same app runs on Linux/Windows	jar deploy
7	Compiler	Converts .java → .class	1.0+	Build pipeline	javac
8	Interpreter	Executes bytecode line-by-line	1.0+	JVM execution stage	JVM
9	JIT Compiler	Converts hot bytecode → native	1.0+	Performance optimization	HotSpot JIT
10	HotSpot	Oracle JVM impl with JIT & GC	1.3+	Production backend servers	OpenJDK HotSpot
11	Class	Blueprint of objects	1.0+	Entity/model/service	class User{}
12	Object	Runtime instance of class	1.0+	Represents data	new User()
13	Method	Function inside class	1.0+	Business operations	createUser()
14	Constructor	Initializes object	1.0+	Creating DTOs/entities	User(){}

15	Default Constructor	Auto constructor if none exists	1.0+	Framework object creation	new User()
16	Parameterized Constructor	Constructor with args	1.0+	Dependency setup	User(id)
17	Field	Member variable	1.0+	Data in entity	private int id;
18	Local Variable	Var inside method	1.0+	Temp calculations	int sum=0;
19	Static Variable	Class-level shared variable	1.0+	Global constants	static int count;
20	Instance Variable	Object-level variable	1.0+	Per-user data	this.name
21	Primitive Types	int, long, char, boolean etc.	1.0+	Fast operations	int a=5;
22	Reference Type	Object reference types	1.0+	Collections, entities	String s;
23	Wrapper Class	Object representation of primitive	1.0+	Generics, collections	Integer x
24	Autoboxing	primitive → wrapper conversion	5+	List operations	List<Integer>
25	Unboxing	wrapper → primitive	5+	Arithmetic operations	int x=Integer
26	OOP	Object oriented programming	1.0+	Enterprise design	MVC
27	Encapsulation	Hide data using access modifiers	1.0+	Secure models	private + getter
28	Abstraction	Hide complexity	1.0+	Interface-based design	PaymentGateway
29	Inheritance	Child gets parent properties	1.0+	Code reuse	extends

30	Polymorphism	Same call, different behavior	1.0+	Strategy impl	pay()
31	Method Overloading	Same name diff params	1.0+	Utils/APIs	add(int)
32	Method Overriding	Redefine parent method	1.0+	Framework hooks	toString()
33	Compile-time Polymorphism	Achieved by overloading	1.0+	Static dispatch	overloaded call
34	Runtime Polymorphism	Achieved by overriding	1.0+	Dynamic dispatch	@Override
35	Interface	Contract, defines behavior	1.0+	Plug-in architecture	implements
36	Abstract Class	Partial implementation	1.0+	Base service, templates	abstract
37	Concrete Class	Fully implemented class	1.0+	Service impl	UserServiceImpl
38	this	Refers to current object	1.0+	Disambiguate variables	this.id=id
39	super	Refers to parent class	1.0+	Parent constructor call	super()
40	static	Belongs to class	1.0+	Utility methods/constants	static final
41	final	Cannot change/override/extend	1.0+	Constants/security	final int
42	Access Modifier	public/private/protected/default	1.0+	Encapsulation	private
43	public	Accessible everywhere	1.0+	Public APIs	public class

44	private	Accessible only inside class	1.0+	Data security	private field
45	protected	Access in package + subclasses	1.0+	Framework extensions	subclass access
46	default access	package-private	1.0+	Internal modules	class X{}
47	Package	Namespace for classes	1.0+	Organize microservices	com.app.user
48	Import	Bring other classes into file	1.0+	Code compilation	import java.util.*;
49	Fully Qualified Name	pkg + class	1.0+	Avoid naming conflicts	java.util.Date
50	Main Method	Entry point of Java program	1.0+	Start app	public static void main
51	Command-line args	Input passed to main	1.0+	Runtime configs	args[0]
52	JVM Arguments	Flags to tune runtime	1.0+	Prod performance	-Xmx2g
53	Heap	Memory for objects	1.0+	Stores runtime objects	new uses heap
54	Stack	Memory for method calls	1.0+	Recursion, locals	stack frames
55	Stack Frame	Each method call context	1.0+	Debugging/performance	method local vars
56	Method Area	Class metadata storage	1.0+	Class loading	class info
57	Metaspace	Native memory for class metadata	8+	Replace PermGen	-XX:MaxMetaspaceSize
58	PermGen	Old class metadata memory area	≤7	Legacy tuning	removed in 8

59	Garbage Collection	Auto cleanup unused objects	1.0+	Memory management	GC cycles
60	GC Root	Reference starting points	1.0+	Determine reachable objects	static, stack
61	Mark-and-Sweep	GC algorithm	1.0+	Identify dead objects	mark live
62	Minor GC	Young gen collection	1.0+	Frequent cleanup	Eden cleanup
63	Major GC	Old gen collection	1.0+	Heavy cleanup	Tenured cleanup
64	Full GC	Entire heap cleanup	1.0+	Dangerous pause	stop-the-world
65	Stop-The-World	JVM pauses threads during GC	1.0+	Latency issues	Full GC pause
66	ClassLoader	Loads classes dynamically	1.0+	Spring loads beans	AppClassLoader
67	Bootstrap ClassLoader	Loads core Java classes	1.0+	Load java.lang.*	rt.jar
68	Extension ClassLoader	Loads ext libs (legacy)	1.0+	Legacy JVM ext	Deprecated
69	Application ClassLoader	Loads app classes	1.0+	Load your code	classpath
70	Classpath	Path containing .class/jar	1.0+	Dependency resolution	-cp
71	Jar	Archive of classes/resources	1.0+	Deploy services	java -jar
72	Manifest	Metadata inside jar	1.0+	Define main class	Main-Class:
73	Exception	Abnormal program event	1.0+	Error handling APIs	try-catch

74	Checked Exception	Must handle at compile time	1.0+	File/DB exceptions	IOException
75	Unchecked Exception	Runtime exception	1.0+	Validation/null errors	NPE
76	Error	JVM/system serious issue	1.0+	OutOfMemory, StackOverflow	Error
77	try-catch	Exception handling block	1.0+	Prevent crash	catch(Exception e)
78	finally	Executes always	1.0+	Resource close	DB close
79	throw	Create exception manually	1.0+	Business rules	throw new Illegal...
80	throws	Declares exception	1.0+	API signature	throws IOException
81	Call Stack	All active method calls	1.0+	Debug issues	stacktrace
82	StackTrace	List of method calls causing error	1.0+	Debugging prod errors	e.printStackTrace()
83	NullPointerException	Accessing method/field on null	1.0+	Frequent bug	obj.method()
84	ClassCastException	Wrong type casting	1.0+	Generics missing	(String) obj
85	ArrayIndexOutOfBoundsException	Invalid array index	1.0+	Bugs in loops	arr[10]
86	ArithmeticException	Divide by zero etc.	1.0+	Calculation validation	/0
87	OutOfMemoryError	Heap space exhausted	1.0+	Memory leak	Java heap space
88	StackOverflowError	Deep recursion	1.0+	Bad recursion	recursion crash

89	String	Immutable character sequence	1.0+	JSON, logs, data	"hello"
90	String Pool	Special memory for literals	1.0+	Save memory	"abc" reused
91	StringBuilder	Mutable string, fast	1.5+	Building JSON response	Append
92	StringBuffer	Thread-safe mutable string	1.0+	Rare now	synchronized ops
93	toString()	Converts object → string	1.0+	Logging/debugging	System.out.println(obj)
94	equals()	Compare object contents	1.0+	User comparisons	a.equals(b)
95	hashCode()	Hash value for map/set	1.0+	HashMap key storage	hashCode()
96	== operator	Reference comparison (objects)	1.0+	Identity check	a==b
97	instanceof	Type check operator	1.0+	Polymorphism handling	obj instanceof User
98	Type Casting	Convert type explicitly	1.0+	Data conversion	(int) d
99	Upcasting	Child → parent reference	1.0+	Polymorphic calls	Animal a=new Dog()
100	Downcasting	Parent → child reference	1.0+	Access child methods	(Dog)a
101	Compile Time	During compilation	1.0+	Catch syntax errors	javac
102	Runtime	During execution	1.0+	Catch runtime errors	NPE
103	Token	Smallest element in source code	1.0+	Parsing compilation	int

104	Lexical Analysis	Convert chars → tokens	1.0+	Compiler stage	tokenization
105	Syntax Analysis	Parse tokens into structure	1.0+	Detect syntax errors	parsing
106	Semantic Analysis	Meaning/typing validation	1.0+	Type checking	assign mismatch
107	Bytecode Verification	JVM ensures safe bytecode	1.0+	Prevent malicious bytecode	verifier
108	Native Code	Machine code executed by CPU	1.0+	High speed exec	JIT output
109	JNI	Call native C/C++ from Java	1.0+	Hardware libs, crypto libs	native
110	Native Method	Java method implemented in C	1.0+	OS/hardware integration	native void
111	Serialization	Convert object → bytes	1.0+	Cache/session /storage	Serializable
112	Deserialization	bytes → object	1.0+	Read stored data	ObjectInputStream
113	Serializable	Marker interface for serialization	1.0+	Cache objects	implements Serializable
114	transient	Exclude field from serialization	1.0+	Hide secrets	transient pass
115	Marker Interface	Interface without methods	1.0+	Metadata tagging	Serializable
116	Enum	Fixed set of constants	5+	Status types	enum Status{}
117	Annotation	Metadata for compile/runtime	5+	Spring/JPA	@Entity

118	@Override	Marks overriding method	5+	Avoid mistake	@Override
119	@Deprecated	Marks outdated API	5+	Prevent usage	@Deprecated
120	@SuppressWarnings	Suppress compile warnings	5+	Clean builds	@SuppressWarnings("unchecked")

✅ **PART 1 complete (1–120)**

Next Parts Plan (to reach 1000+)

I will generate the remaining terms like this:

- ✅ PART 2: 121–240 (**Collections + Generics + Data Structures**)
- ✅ PART 3: 241–360 (**Multithreading + Concurrency**)
- ✅ PART 4: 361–480 (**IO, NIO, Files, Networking**)
- ✅ PART 5: 481–600 (**JDBC, SQL, Transactions, ORM terms**)
- ✅ PART 6: 601–720 (**Java 8 Functional, Streams, Lambda ecosystem**)
- ✅ PART 7: 721–840 (**Java 9–21: Modules, Records, Sealed, Virtual Threads**)
- ✅ PART 8: 841–1000+ (**JVM tuning, GC, monitoring, tools, security**)

If you say **"Continue Part 2"**, I will generate **Terms 121–240** next.

Here is:

✅ **PART 2 (Terms 121–240) — Collections Framework + Generics + Data Structures (Real-Time Use + Examples)**

✅ **Java 1000+ Terminology MASTER LIST (PART 2: 121–240)**

#	Term	Meaning (Detailed)	Version/ Era	Real-Time Use Case	Example
121	Collections Framework	Standard library for storing/managin	1.2+	API results, caching, DB loaded data	java.util.*

		g groups of objects			
122	Collection	Root interface for most containers	1.2+	Generic storage abstraction	Collection<String>
123	List	Ordered collection (allows duplicates)	1.2+	API response lists, UI lists	List<User>
124	Set	Unordered unique elements	1.2+	Unique IDs, permissions	Set<String>
125	Map	Key-value storage	1.2+	Lookup by ID, caching	Map<Long,User>
126	Iterator	Traversal mechanism for collections	1.2+	Safe iteration	it.hasNext()
127	Iterable	Enables foreach loop	1.5+	Custom collection loops	implements Iterable
128	Enhanced for loop	For-each loop over iterable	5+	Cleaner iteration	for(User u: list)
129	Array	Fixed-size indexed structure	1.0+	High-performance data	int[] a
130	Dynamic Array	Auto-growing array behavior	1.2+	List like ArrayList	ArrayList
131	ArrayList	Resizable array implementation of List	1.2+	Frequent reads, random access	new ArrayList<>()
132	LinkedList	Doubly-linked list implementation	1.2+	Frequent insert/delete in middle	new LinkedList<>()
133	Vector	Thread-safe resizable array (legacy)	1.0+	Legacy code only	new Vector<>()
134	Stack (class)	LIFO stack (legacy Vector-	1.0+	Legacy stack operations	new Stack<>()

		based)			
135	Deque	Double ended queue interface	1.6+	Stack+Queue modern design	Deque<Integer>
136	ArrayDeque	Fast resizable deque	1.6+	Modern stack/queue replacement	new ArrayDeque<>()
137	Queue	FIFO structure	1.5+	Job scheduling, message buffering	Queue<Task>
138	PriorityQueue	Queue ordered by priority	1.5+	Scheduling, Dijkstra algorithm	new PriorityQueue<>()
139	BlockingQueue	Thread-safe queue with blocking ops	5+	Producer-consumer systems	LinkedBlockingQueue
140	LinkedBlockingQueue	BlockingQueue using linked nodes	5+	Thread pool work queues	new LinkedBlockingQueue<>()
141	ArrayBlockingQueue	Fixed-size blocking queue	5+	Backpressure control	new ArrayBlockingQueue<>(100)
142	ConcurrentLinkedQueue	Lock-free thread-safe queue	5+	High throughput messaging	new ConcurrentLinkedQueue<>()
143	CopyOnWriteArrayList	Thread-safe list using copy-on-write	5+	Many reads, few writes (listeners)	new CopyOnWriteArrayList<>()
144	CopyOnWriteArraySet	Thread-safe set using copy-on-write	5+	Read-heavy unique elements	new CopyOnWriteArraySet<>()
145	HashSet	Hash-based Set	1.2+	Unique values	new HashSet<>()
146	LinkedHashSet	Maintains insertion order	1.4+	Ordered unique list	new LinkedHashSet<>()
147	TreeSet	Sorted set (RB Tree)	1.2+	Sorted unique values	new TreeSet<>()

148	EnumSet	Highly efficient Set for enums	5+	Permission sets	EnumSet.of(A,B)
149	HashMap	Hash-based key/value map	1.2+	Cache, fast lookup	new HashMap<>()
150	LinkedHashMap	Maintains insertion order in Map	1.4+	LRU cache design	new LinkedHashMap<>()
151	TreeMap	Sorted map based on keys	1.2+	Sorted reporting (timestamps)	new TreeMap<>()
152	Hashtable	Legacy synchronized Map	1.0+	Legacy apps only	new Hashtable<>()
153	ConcurrentHashMap	Thread-safe high performance map	5+	Multi-thread caching/session store	new ConcurrentHashMap<>()
154	WeakHashMap	Keys removed when no strong ref	1.2+	Memory-sensitive caches	new WeakHashMap<>()
155	IdentityHashMap	Uses == instead of equals	1.4+	Rare: object identity mapping	new IdentityHashMap<>()
156	NavigableMap	Map with navigation methods	1.6+	Range queries	floorKey()
157	NavigableSet	Sorted set navigation support	1.6+	Range operations	higher()
158	SortedMap	Keys sorted	1.2+	Ordered data	SortedMap
159	SortedSet	Elements sorted	1.2+	Ranking lists	SortedSet
160	Map.Entry	Key-value pair in map	1.2+	Iterating map content	for(var e: map.entrySet())
161	entrySet()	Returns set of entries	1.2+	Map iteration fastest	map.entrySet()

162	keySet()	Returns set of keys	1.2+	Validate keys	map.keySet()
163	values()	Returns collection of values	1.2+	Aggregate values	map.values()
164	Hashing	Converting object → bucket index	1.2+	Performance in HashMap/Set	hashCode()
165	hashCode() contract	Equal objects must have same hash	1.0+	Prevent Map bugs	equals/hashcode
166	Collision	Two keys map to same bucket	1.2+	Performance tuning	hash collision
167	Bucket	Storage slot inside HashMap	1.2+	Internal structure	bucket array
168	Load Factor	Threshold for resizing	1.2+	Optimize memory/performance	default 0.75
169	Rehashing	Recomputing buckets on resize	1.2+	HashMap grows	internal operation
170	Initial Capacity	Starting bucket array size	1.2+	Performance optimization	new HashMap<>(256)
171	Time Complexity	Growth of algorithm operations	CS	Performance critical code	O(1), O(n)
172	Big-O Notation	Complexity notation	CS	Choose best data structures	O(log n)
173	Fail-fast Iterator	Throws exception on modification	1.2+	Detect concurrent modification	ConcurrentModificationException
174	Fail-safe Iterator	Doesn't throw, works on copy/structure	5+	Concurrent read iteration	ConcurrentHashMap

175	ConcurrentModificationException	Occurs when collection modified during iteration	1.2+	Debug iteration bugs	modifying list in loop
176	Generics	Type-safe parameterization	5+	Avoid casting errors	List<String>
177	Raw Type	Collection without generic types	1.2 legacy	Avoid in enterprise	List list
178	Type Erasure	Generics removed at runtime	5+	Limits runtime generic reflection	runtime sees List
179	Generic Method	Method with type parameters	5+	Utility library methods	<T> T get()
180	Generic Class	Class with type parameter	5+	Reusable container	class Box<T>
181	Type Parameter	Placeholder type like T, E, K, V	5+	Generic design	<T>
182	Bounded Type	Restrict types using extends	5+	Numeric calculation generics	<T extends Number>
183	Upper Bounded Wildcard	? extends Type	5+	Read-only flexible APIs	List<? extends Number>
184	Lower Bounded Wildcard	? super Type	5+	Write-safe generics	List<? super Integer>
185	Unbounded Wildcard	? any type	5+	Unknown type APIs	List<?>
186	PECS	Producer Extends Consumer Super rule	5+	Generic API correctness	? extends / ? super
187	Comparator	Custom sorting logic	1.2+	Sort objects in reports	Comparator.comparing()

188	Comparable	Natural ordering interface	1.2+	Default sorting	implements Comparable
189	compareTo()	Method for Comparable	1.2+	Sorting inside TreeSet/Tree Map	a.compareTo(b)
190	compare()	Comparator method	1.2+	External sorting strategy	(a,b)->...
191	Natural Order	Default sorting order	1.2+	Strings alphabet order	"a" < "b"
192	Custom Order	Sorting as per business logic	1.2+	Salary descending, date sorting	Comparator
193	Stable Sort	Maintains original order for equals	concept	UI sorting, reports	stable ordering
194	Unmodifiable Collection	Cannot modify after creation	1.2+	Protect configuration	Collections.unmodifiableList()
195	Immutable Collection	Truly immutable collection	9+	Config constants	List.of()
196	Collections Utility Class	Helper methods for collections	1.2+	Sorting, sync wrappers	Collections.sort()
197	Arrays Utility Class	Helper for arrays	1.2+	Sorting/searching arrays	Arrays.sort()
198	Collections.sort()	Sorts list using natural/comparator	1.2+	Sorting API output	Collections.sort(list)
199	Arrays.sort()	Sort array	1.2+	Sorting primitives fast	Arrays.sort(arr)
200	Binary Search	Search in sorted structure	1.2+	Fast query lookup	Collections.binarySearch()
201	Collections.binarySearch()	Search in sorted list	1.2+	Search UI results	binarySearch(list,k)
202	Arrays.binarySearch()	Search in sorted array	1.2+	Performance search	Arrays.binarySearch(arr,x)

203	Collections.synchronizedList()	Thread-safe wrapper list	1.2+	Legacy concurrency	synchronizedList(new ArrayList<>())
204	Collections.synchronizedMap()	Thread-safe wrapper map	1.2+	Legacy concurrency	synchronizedMap(map)
205	Collections.synchronizedSet()	Thread-safe wrapper set	1.2+	Legacy concurrency	synchronizedSet(set)
206	Random Access	Fast index access marker	1.4+	Optimize algorithms	ArrayList implements RandomAccess
207	RandomAccess Interface	Marker for index-fast lists	1.4+	Detect list type	instanceof RandomAccess
208	Spliterator	Splitting iterator for parallelism	8+	Parallel stream processing	list.spliterator()
209	Stream	Sequence of elements with operations	8+	Data pipeline	list.stream()
210	Terminal Operation	Ends stream pipeline	8+	Collect results	collect()
211	Intermediate Operation	Lazy transformation step	8+	filter/map chain	map(), filter()
212	Lazy Evaluation	Executes only at terminal	8+	Performance efficiency	stream laziness
213	Collector	Reduce stream into structure	8+	Convert to list/map	Collectors.toList()
214	Collectors	Factory class for collectors	8+	Grouping, joining	Collectors.groupingBy()
215	groupingBy()	Group stream by key	8+	Reports, analytics	groupingBy(User::role)
216	partitioningBy()	Split into 2 groups boolean	8+	Active/inactive split	partitioningBy(User::active)

217	mapping() Collector	Map before collecting	8+	Extract fields	mapping(User::name,toList())
218	joining()	Join strings into 1	8+	CSV generation	joining(",")
219	toMap()	Convert stream to map	8+	Keyed results	Collectors.toMap()
220	Optional	Container to avoid null	8+	API responses safe	Optional.ofNullable(x)
221	Optional.map()	Transform value if present	8+	Null-safe mapping	opt.map()
222	Optional.orElse()	Default if not present	8+	Fallback response	orElse("NA")
223	Optional.orElseGet()	Supplier fallback lazy	8+	Expensive fallback computed only if needed	orElseGet()
224	Optional.orElseThrow()	Throw exception if empty	8+	Mandatory field validation	orElseThrow()
225	Null-safe programming	Avoid null issues patterns	modern	Prevent NPE in prod	Optional
226	Defensive Copy	Create new copy to prevent mutation	1.0+	Secure immutability in DTOs	new ArrayList<>(list)
227	Shallow Copy	Copies references, not deep data	1.0+	Fast clone for simple objects	clone()
228	Deep Copy	Full object graph copy	concept	Safe copying nested data	serialize copy
229	Cloneable	Marker interface for clone()	1.0+	Rare in enterprise	implements Cloneable
230	clone()	Creates copy of object	1.0+	Object duplication	obj.clone()
231	Immutable Design	Class cannot change state	1.0+	Thread safety, caching	String

232	Mutable Collection	Collection that can change	1.2+	CRUD operations	list.add()
233	Immutable Factory Methods	List.of/Set.of/Map.of	9+	Config constant collections	List.of(1,2)
234	List.of()	Create immutable list	9+	Constant config	List.of("A","B")
235	Set.of()	Create immutable set	9+	Constants	Set.of("READ","WRITE")
236	Map.of()	Create immutable map	9+	Key-value constants	Map.of("k","v")
237	Map.ofEntries()	Immutable map using entries	9+	Large immutable maps	Map.ofEntries(...)
238	Entry	Key/value pair factory	9+	Used in ofEntries	Map.entry("k","v")
239	Concurrent Collection	Thread-safe data structures	5+	Multi-thread microservices	ConcurrentHashMap
240	Data Structure	Organized storage for data	CS	Choose for performance	List/Map/Set

---

✅ **PART 2 complete (121–240)**

If you say **"PART 3"**, I will generate:

✅ **PART 3 (241–360) — Multithreading + Concurrency + Executor Framework**

This will include:

- Thread lifecycle terms
- ExecutorService / Callable / Future
- Locks, monitor, deadlocks
- Java Memory Model terms
- Atomic variables
- ThreadLocal / InheritableThreadLocal
- ForkJoinPool

- CompletableFuture advanced terms

Now here is:

✅ PART 3 (Terms 241–360) — Multithreading + Concurrency + Executor Framework (Real-Time Use + Examples)

This part includes **core threading terminology**, **Executor framework**, **concurrency utilities**, and **JMM basics** used in real enterprise backend systems.

✅ Java 1000+ Terminology MASTER LIST (PART 3: 241–360)

#	Term	Meaning (Detailed)	Version/Era	Real-Time Use Case	Example
241	Multithreading	Running multiple threads concurrently	1.0+	Handle multiple requests/tasks	Web server threads
242	Thread	Lightweight execution unit	1.0+	Async processing	new Thread()
243	Process	OS-level running program containing threads	OS	JVM process hosting threads	java app.jar
244	Concurrency	Multiple tasks progress at once	1.0+	Web servers	1000 req/sec
245	Parallelism	Tasks executing simultaneously on multiple cores	1.0+	Batch computations	parallel streams
246	Thread Lifecycle	States: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED	1.0+	Debugging deadlocks	thread.getState()
247	NEW state	Thread created but not started	1.0+	Controlled start	new Thread()
248	RUNNABLE	Ready/running	1.0+	Active request	after start()

	state	on CPU		handling	
249	BLOCKED state	Waiting for monitor lock	1.0+	Contention analysis	sync blocked
250	WAITING state	Waiting indefinitely	1.0+	wait()/join()	wait()
251	TIMED_WAITING	Waiting for fixed time	1.0+	timeouts	sleep(1000)
252	TERMINATED	Thread finished execution	1.0+	Logging completion	after run ends
253	start()	Starts thread, creates new call stack	1.0+	Run async task	t.start()
254	run()	Task code executed by thread	1.0+	Business logic	public void run()
255	sleep()	Pause execution for time	1.0+	Retry/backoff logic	Thread.sleep(200)
256	yield()	Hint scheduler to give CPU to others	1.0+	Rare tuning/testing	Thread.yield()
257	join()	Wait for another thread to finish	1.0+	Batch jobs	t.join()
258	interrupt()	Signal thread to stop/wake	1.0+	Shutdown handling	t.interrupt()
259	InterruptedException	Thrown when interrupted in waiting	1.0+	Proper shutdown	catch interrupt
260	Daemon Thread	Background thread that doesn't block JVM exit	1.0+	Monitoring, cleanup	setDaemon(true)
261	User Thread	Normal thread keeps JVM alive	1.0+	Main service worker threads	default

262	Thread Priority	Scheduling hint (1–10)	1.0+	Rare usage	setPriority(10)
263	Context Switching	CPU switches between threads	OS	Performance tuning	reduce threads
264	Race Condition	Wrong output due to timing between threads	1.0+	Payments double debit	shared balance
265	Critical Section	Code needing exclusive access	1.0+	Update shared cache	sync block
266	Synchronization	Ensuring thread-safe access	1.0+	Shared data updates	Synchronized
267	synchronized keyword	Lock-based mutual exclusion	1.0+	Protect shared resources	synchronized(obj){}
268	Monitor Lock	Intrinsic lock used by synchronized	1.0+	Thread coordination	object monitor
269	Intrinsic Lock	Built-in object lock	1.0+	Simple locking	sync methods
270	Mutual Exclusion	Only 1 thread in critical section	1.0+	Prevent data corruption	lock
271	Deadlock	Two threads waiting forever for each other	1.0+	Banking/Inventory bug	lock ordering
272	Livelock	Threads active but not progressing	1.0+	retry loops	both keep yielding
273	Starvation	Thread never gets CPU/lock	1.0+	Priority issues	low priority
274	Thread Safety	Code safe under concurrent access	1.0+	Web apps	stateless services
275	Thread-safe class	Class designed for concurrency	1.0+	ConcurrentHashMap	safe map
276	Non-thread-	Breaks in	1.0+	ArrayList	race

	safe class	concurrency		under writes	
277	Reentrancy	Same thread can acquire lock multiple times	1.0+	Nested lock calls	sync recursion
278	ReentrantLock	Explicit lock with more features	5+	Try-lock/timeouts	new ReentrantLock()
279	lock()	Acquire explicit lock	5+	Controlled sync	lock.lock()
280	unlock()	Release explicit lock	5+	Avoid deadlocks	lock.unlock()
281	tryLock()	Attempts lock without blocking	5+	Avoid deadlock, degrade gracefully	lock.tryLock()
282	ReadWriteLock	Separate read/write locks	5+	Cache many reads few writes	ReentrantReadWriteLock
283	ReentrantReadWriteLock	Implementation of ReadWriteLock	5+	Shared configs	rw.readLock()
284	Condition	Lock-based wait/notify alternative	5+	Producer-consumer control	condition.await()
285	wait()	Releases monitor and waits	1.0+	Producer-consumer legacy	obj.wait()
286	notify()	Wakes single waiting thread	1.0+	Signal waiting consumer	obj.notify()
287	notifyAll()	Wakes all waiting threads	1.0+	Multi consumers	obj.notifyAll()
288	Spurious Wakeup	Thread wakes without notify	1.0+	Must use while loop	while(!cond)wait()
289	volatile keyword	Visibility guarantee, prevents caching	1.0+	Stop flag in threads	volatile boolean stop

290	Visibility	Changes in one thread seen by others	JMM	Fix stale values	volatile/sync
291	Atomicity	Operation occurs fully or not at all	JMM	Prevent partial updates	atomic variables
292	Ordering	Execution order constraints across threads	JMM	Fix reordering issues	happens-before
293	Java Memory Model (JMM)	Defines how threads interact via memory	5+ clarified	Correct concurrent programming	happens-before
294	Happens-Before	Guarantee visibility/order	JMM	Ensure correct reads/writes	sync establishes
295	Atomic Variables	Lock-free thread-safe primitives	5+	Counters/metrics	AtomicInteger
296	AtomicInteger	Atomic int operations	5+	Request counter	incrementAndGet()
297	AtomicLong	Atomic long operations	5+	IDs/metrics	getAndAdd()
298	AtomicBoolean	Atomic boolean operations	5+	Flags	compareAndSet()
299	CAS	Compare-And-Swap, lock-free update	5+	High performance concurrency	atomic CAS
300	compareAndSet()	CAS update method	5+	Only update if expected value	cas
301	LongAdder	Faster than AtomicLong under contention	8+	High traffic counters	adder.increment()
302	DoubleAdder	Like LongAdder for doubles	8+	Metrics tracking	doubleAdder.add()
303	Executor	Thread pool	5+	Microservices	executors

	Framework	based task execution		async workloads	
304	Executor	Simple task execution interface	5+	Generic async dispatch	execute()
305	ExecutorService	Executor with lifecycle & futures	5+	Background jobs	submit()
306	Thread Pool	Reused threads to execute tasks	5+	API request processing	Tomcat pool
307	Executors class	Factory for executors	5+	Quick thread pool creation	Executors.newFixedThreadPool()
308	newFixedThreadPool	Fixed size thread pool	5+	Limited concurrency tasks	newFixedThreadPool(10)
309	newCachedThreadPool	Dynamic pool grows as needed	5+	Short-lived burst tasks	cached pool
310	newSingleThreadExecutor	Single thread executor	5+	Sequential job execution	1 thread
311	ScheduledExecutorService	Executor with scheduling	5+	Cron-like tasks	scheduleAtFixedRate()
312	scheduleAtFixedRate	Schedule periodic execution fixed rate	5+	Health check, polling	every 5 sec
313	scheduleWithFixedDelay	Schedule next after delay post completion	5+	Safe periodic tasks	delay after run
314	Runnable	Task with no return	1.0+	Fire-and-forget tasks	() -> log()
315	Callable	Task with return and exception	5+	Async results	Callable<Integer>
316	Future	Represents async result placeholder	5+	Get result later	future.get()

317	future.get()	Blocks until result available	5+	Wait for remote call	get()
318	Timeout	Fail if task takes too long	5+	Prevent stuck requests	get(2,SECONDS)
319	Cancellation	Stop submitted task	5+	User cancels report	future.cancel(true)
320	Shutdown	Stop accepting new tasks	5+	Graceful service close	shutdown()
321	shutdownNow()	Attempts to interrupt running tasks	5+	Hard stop service	shutdownNow()
322	awaitTermination	Wait for pool shutdown	5+	Cleanup at shutdown	awaitTermination()
323	ThreadFactory	Customize thread creation	5+	Naming threads in logs	new ThreadFactory()
324	RejectedExecutionHandler	Handle tasks rejected by pool	5+	Backpressure strategy	AbortPolicy
325	AbortPolicy	Reject by throwing exception	5+	Fail fast	default
326	CallerRunsPolicy	Calling thread runs task	5+	Slow producer, backpressure	caller runs
327	DiscardPolicy	Drop task silently	5+	Non-critical tasks	discard
328	DiscardOldestPolicy	Drop oldest queued task	5+	Real-time streaming	discard old
329	ForkJoin Framework	Divide tasks into subtasks	7+	Parallel computation	fork/join
330	ForkJoinPool	Pool for fork/join tasks	7+	Parallel streams execution	common pool
331	Work Stealing	Idle threads steal tasks	7+	High throughput	fork join
332	RecursiveTask	ForkJoin task	7+	Parallel sum	extends

		returning value			RecursiveTask
333	RecursiveAction	ForkJoin task no return	7+	Parallel processing	extends RecursiveAction
334	CompletableFuture	Powerful async pipeline	8+	Parallel service calls	thenApply()
335	CompletionStage	Interface for async stages	8+	Async DSL	CompletionStage
336	supplyAsync()	Run supplier async	8+	Async fetch	supplyAsync(()->call())
337	thenApply()	Transform result async	8+	Map response	.thenApply()
338	thenCompose()	Flatten nested futures	8+	Dependent async calls	.thenCompose()
339	thenCombine()	Combine two futures	8+	Merge results from 2 services	.thenCombine()
340	allOf()	Wait for all tasks	8+	Batch parallel calls	allOf(f1,f2)
341	anyOf()	First completed future	8+	Fastest response fallback	anyOf()
342	exceptionally()	Handle exception in pipeline	8+	fallback results	.exceptionally(ex->...)
343	handle()	Handle success+failure both	8+	advanced fallback	.handle()
344	complete()	Manually set result	8+	manual control	future.complete()
345	completeExceptionally()	Manually set exception	8+	custom error	completeExceptionally()
346	ThreadLocal	Thread-specific storage	1.2+	User/request context	ThreadLocal<User>
347	InheritableThr	Child thread	1.2+	Logging trace	inheritance

	eadLocal	inherits parent's value		id	
348	Context Propagation	Passing request info across threads	modern	tracing/logging	MDC
349	MDC	Mapped Diagnostic Context logging	logback/log4j	Request tracing	traceId
350	Semaphore	Limit concurrent access count	5+	Rate limiting	new Semaphore(10)
351	CountDownLatch	Wait until N operations done	5+	Wait for services init	latch.await()
352	CyclicBarrier	Threads meet at barrier repeatedly	5+	Batch parallel stages	barrier.await()
353	Phaser	More flexible barrier/latch	7+	Multi-phase workflows	phaser.arrive()
354	Exchanger	Two threads exchange data	5+	Pipeline stage exchange	ex.exchange()
355	Synchronized Queue	Thread-safe queue via sync	legacy	Producer-consumer	Collections.synchronizedList
356	Thread Dump	Snapshot of threads state	JVM	Debug deadlocks/high CPU	Jstack
357	jstack	Tool to print thread dump	JDK	Prod debugging	jstack <pid>
358	Deadlock Detection	Finding cyclic lock dependencies	JVM tools	Debug stuck server	thread dump
359	Contention	Multiple threads competing for lock	JVM	Performance tuning	reduce locking
360	Throughput vs Latency	Throughput=ops/sec, latency=response time	perf	Microservice optimization	tuning pools

---

✅ **PART 3 complete (241–360)**

If you say **"PART 4"**, I'll generate:

✅ **PART 4 (361–480) — IO + NIO + File Handling + Networking + Serialization (enterprise terms)**

It will include:

- Streams (InputStream/OutputStream/Reader/Writer)
- Buffered streams
- NIO Channels/Buffers/Selectors
- Files/Paths/WatchService
- Socket/ServerSocket/HTTP Client terms
- Serialization protocols and best practices

Now here is:

✅ **PART 4 (Terms 361–480) — Java IO + NIO + File Handling + Networking + Serialization (Real-Time Use + Examples)**

This part is **extremely important in real enterprise software**, especially:

- file upload/download modules
- log processing
- ETL pipelines
- network communication
- socket programming
- serialization / data transfer

✅ **Java 1000+ Terminology MASTER LIST (PART 4: 361–480)**

#	Term	Meaning (Detailed)	Version/ Era	Real-Time Use Case	Example
361	IO	Input/Output operations (files/network/streams)	1.0+	File read/write, data transfer	java.io
362	Stream	Continuous flow	1.0+	Read	InputStream

		of data bytes/chars		file/network response	
363	Byte Stream	Stream working with bytes	1.0+	Images, PDFs, ZIP	InputStream
364	Character Stream	Stream working with chars (Unicode)	1.0+	Text files, JSON	Reader
365	InputStream	Base class for byte input	1.0+	Read binary data	FileInputStream
366	OutputStream	Base class for byte output	1.0+	Write binary data	FileOutputStream
367	Reader	Base class for char input	1.1+	Read text	FileReader
368	Writer	Base class for char output	1.1+	Write text	FileWriter
369	File	Abstract file path representation	1.0+	File operations legacy	new File("a.txt")
370	FileInputStream	Reads bytes from file	1.0+	Read image/pdf	new FileInputStream()
371	FileOutputStream	Writes bytes to file	1.0+	Write binary output	new FileOutputStream()
372	FileReader	Reads file chars	1.1+	Read config text	new FileReader()
373	FileWriter	Writes chars to file	1.1+	Write logs/report	new FileWriter()
374	BufferedInputStream	Buffered byte reading	1.0+	Faster file reads	wrap InputStream
375	BufferedOutputStream	Buffered byte writing	1.0+	Faster file writes	wrap OutputStream
376	BufferedReader	Buffered char reading + readLine()	1.1+	Read large text files	readLine()
377	BufferedWrite	Buffered char	1.1+	Write large	write()

	r	writing		reports	
378	ByteArrayInputStream	Read from byte array	1.0+	Testing, in-memory IO	new ByteArrayInputStream()
379	ByteArrayOutputStream	Write to byte array	1.0+	Build bytes for upload	toByteArray()
380	DataInputStream	Reads primitive types	1.0+	Binary protocols	readInt()
381	DataOutputStream	Writes primitive types	1.0+	Binary protocols	writeInt()
382	ObjectInputStream	Read serialized objects	1.0+	Deserialize cached object	readObject()
383	ObjectOutputStream	Write serialized objects	1.0+	Serialize objects	writeObject()
384	Serialization	Object → bytes conversion	1.0+	Session storage, caching	Serializable
385	Deserialization	bytes → object conversion	1.0+	Read stored object	readObject()
386	SerialVersionUID	Version ID for serialization compatibility	1.0+	Avoid InvalidClassException	private static final long
387	InvalidClassException	Mismatch SerialVersionUID	1.0+	Deployment serialization issues	UID mismatch
388	transient	Field excluded from serialization	1.0+	Hide secrets/passwords	transient String pwd;
389	Externalizable	Full control over serialization	1.1+	Custom serialization format	writeExternal()
390	writeObject()	Custom serialization hook	1.0+	Encrypt/compress serialization	private method

391	readObject()	Custom deserialization hook	1.0+	Restore state	private method
392	FileNotFound Exception	File missing	1.0+	Validate file upload path	try/catch
393	IOException	General I/O failure	1.0+	Network/file errors	throws IOException
394	try-with-resources	Auto close resources	7+	Prevent leaks	try(...) {}
395	AutoCloseable	Interface enabling auto-close	7+	Custom resources	implements AutoCloseable
396	Closeable	Close I/O resources	1.5+	IO standard closing	close()
397	flush()	Forces buffered output write	1.0+	Ensure data written	writer.flush()
398	FileDescriptor	OS-level file handle	1.0+	Low-level IO	getFD()
399	Charset	Character encoding type	1.4+	UTF-8 conversions	StandardCharsets.UTF_8
400	Encoding	Converting chars to bytes	concept	Prevent garbled text	UTF-8
401	UTF-8	Unicode encoding standard	common	Web systems & APIs	default modern
402	StandardCharsets	Built-in charset constants	1.7+	Avoid typos encoding	UTF_8
403	Scanner	Simple parser for input	5+	Console input, parsing	new Scanner(System.in)
404	Formatter	Format output like printf	5+	Reports/log formatting	String.format()
405	PrintStream	Output stream printing values	1.0+	Logging to console	System.out
406	PrintWriter	Writer for	1.1+	File reports	new PrintWriter()

		formatted output			
407	Standard Input	System input stream	1.0+	CLI tools	System.in
408	Standard Output	System output stream	1.0+	CLI output	System.out
409	Standard Error	Error output stream	1.0+	Error logging	System.err
410	NIO	Non-blocking IO package	1.4+	High-performance IO	java.nio
411	NIO.2	Enhanced file IO	7+	Modern file handling	java.nio.file
412	Buffer	NIO container for data	1.4+	Efficient IO	ByteBuffer
413	ByteBuffer	Buffer for bytes	1.4+	Networking/file channels	ByteBuffer.allocate()
414	CharBuffer	Buffer for chars	1.4+	Text processing	CharBuffer
415	Buffer Position	Current read/write index	1.4+	Buffer state mgmt	position()
416	Buffer Limit	Max accessible index	1.4+	Avoid overflow	limit()
417	Buffer Capacity	Total size of buffer	1.4+	Memory sizing	capacity()
418	flip()	Switch write mode → read mode	1.4+	NIO read after write	buffer.flip()
419	clear()	Reset buffer for writing	1.4+	Reuse buffer	buffer.clear()
420	rewind()	Reset position to 0 (re-read)	1.4+	Replay data	buffer.rewind()
421	Channel	NIO connection	1.4+	File/network	FileChannel

		for IO		operations	
422	FileChannel	Channel for file operations	1.4+	Fast file copy	transferTo()
423	SocketChannel	Channel for socket operations	1.4+	Non-blocking network	SocketChannel.open()
424	ServerSocketChannel	Accept socket connections	1.4+	High-performance server	accept connections
425	DatagramChannel	UDP channel	1.4+	Video/audio, telemetry	UDP packets
426	Selector	Multiplex non-blocking channels	1.4+	High scale server	select loop
427	SelectionKey	Token representing channel registration	1.4+	Event-based IO	OP_READ
428	Non-blocking Mode	IO operations don't block thread	1.4+	High concurrency servers	configureBlocking(false)
429	Blocking Mode	IO waits until operation done	1.0+	Simple file/socket IO	default
430	Path	Modern file path abstraction	7+	Cross-platform paths	Paths.get()
431	Paths	Factory for Path objects	7+	Build file paths	Paths.get("a.txt")
432	Files	Utility class for file ops	7+	Read/write/copy/move	Files.readString()
433	Files.readAllBytes	Reads entire file into byte[]	7+	Load image/pdf	Files.readAllBytes(p)
434	Files.readString	Reads entire file as string	11+	Read json/config quickly	Files.readString(path)
435	Files.write	Writes	7+	Save reports	Files.write(path,

		bytes/string to file			data)
436	Files.copy	Copy file	7+	Backup file operation	Files.copy(src,dst)
437	Files.move	Move/rename file	7+	Archive processed file	Files.move()
438	Files.delete	Delete file	7+	Cleanup temp files	Files.deleteIfExists()
439	FileVisitor	Visit file tree	7+	Scan folders	walkFileTree()
440	SimpleFileVisitor	Convenience class for traversal	7+	Search & delete	extend visitor
441	walkFileTree	Traverse recursively	7+	Cleanup directories	Files.walkFileTree()
442	Files.walk	Stream of paths from traversal	8+	Large directory scanning	Files.walk(path)
443	WatchService	Monitor directory changes	7+	Auto reload configs/uploads	watch folder
444	File Lock	Prevent concurrent writing	1.4+	Multi-process safety	channel.lock()
445	Memory-mapped File	File mapped into memory	1.4+	Huge file processing	MappedByteBuffer
446	MappedByteBuffer	Buffer for memory-mapped files	1.4+	Fast random access	large logs
447	Socket	Endpoint for TCP communication	1.0+	Client-server messaging	new Socket()
448	ServerSocket	Listens for TCP connections	1.0+	Socket server	new ServerSocket(8080)
449	TCP	Reliable	network	HTTP, DB	TCP connection

		connection protocol		communicatio n	
450	UDP	Fast unreliable protocol	network	Streaming, DNS	UDP datagrams
451	Client-Server Model	Client requests, server responds	network	Web apps	REST
452	Port	Endpoint number for service	network	Web server port	8080
453	IP Address	Unique device address	network	Network communicatio n	192.168.x.x
454	DNS	Converts domain name → IP	network	API call resolution	google.com → IP
455	URL	Uniform Resource Locator	network	API endpoints	https://...
456	URI	Identifier, superset of URL	network	Resource IDs	urn:
457	HttpURLConn ection	Old HTTP client	1.1+	Legacy APIs	openConnection()
458	Java 11 HttpClient	Modern HTTP client	11+	REST calls in microservices	HttpClient.newHttp Client()
459	HttpRequest	Java 11 request builder	11+	Call REST API	.GET()
460	HttpResponse	Java 11 response object	11+	Get status/body	.body()
461	HTTP Methods	GET/POST/PUT /DELETE etc	REST	CRUD APIs	GET user
462	GET	Retrieve resource	REST	Fetch data	/users/1
463	POST	Create resource	REST	Create user/order	/users
464	PUT	Update full	REST	Update user	PUT /users/1

		resource			
465	PATCH	Partial update	REST	Update status only	PATCH /users/1
466	DELETE	Remove resource	REST	Delete record	DELETE /users/1
467	HTTP Status Code	Response codes 200/404/500	REST	API debugging	200 OK
468	200 OK	Successful request	REST	Normal API response	200
469	201 Created	Resource created	REST	POST success	201
470	400 Bad Request	Client sent invalid input	REST	Validation failure	400
471	401 Unauthorized	Auth missing/invalid	REST	Token missing	401
472	403 Forbidden	Not allowed	REST	Role restriction	403
473	404 Not Found	Resource missing	REST	Wrong ID	404
474	409 Conflict	Duplicate state conflict	REST	Duplicate email/user	409
475	500 Internal Server Error	Server crash error	REST	Exception occurred	500
476	Timeout (network)	Server doesn't respond on time	network	Prevent hanging calls	connect timeout
477	Connection Pooling	Reuse network connections	modern	High throughput REST calls	keep-alive
478	Keep-Alive	Maintain persistent TCP connection	HTTP	Reduce latency	persistent conn
479	SSL/TLS	Secure encrypted transport	network	Secure APIs	HTTPS

480	HTTPS	HTTP over TLS	network	Secure login/payment	https://

✅ **PART 4 complete (361–480)**

If you say **"PART 5"**, I will generate:

✅ **PART 5 (481–600) — JDBC + SQL + Transactions + ORM + Database Terminology**

It will include:

- JDBC driver types
- Connection / Statement / PreparedStatement
- ResultSet
- Transaction management
- ACID, isolation levels
- Batch updates
- ORM concepts (Hibernate/JPA basics)
- Lazy vs eager loading
- N+1 query problem
- Connection pools (HikariCP etc.)

Now here is:

✅ **PART 5 (Terms 481–600) — JDBC + SQL + Transactions + ORM (JPA/Hibernate) + Connection Pooling**

This section is **pure enterprise Java**, used daily in:

- banking apps
- e-commerce (orders/payments)
- HRMS systems
- inventory systems
- microservices with DB persistence

✅ **Java 1000+ Terminology MASTER LIST (PART 5: 481–600)**

#	Term	Meaning (Detailed)	Version/ Era	Real-Time Use Case	Example
481	Database (DB)	Persistent storage system	Enterprise	Store users/orders/ payments	MySQL/Postgres
482	RDBMS	Relational DB using tables	Enterprise	Transactional apps	MySQL, Oracle
483	SQL	Structured Query Language	Enterprise	CRUD + joins	SELECT * FROM user
484	Table	Rows/columns structure	Enterprise	User table, Order table	users
485	Row / Record	One entry in table	Enterprise	One user record	id=1
486	Column / Field	Attribute of table	Enterprise	email, name	email VARCHAR
487	Primary Key (PK)	Unique identifier of record	Enterprise	User id/order id	id BIGINT PK
488	Foreign Key (FK)	Relationship key referencing another table	Enterprise	user_id in orders	FK user(id)
489	Constraint	DB rule (PK, FK, UNIQUE)	Enterprise	Prevent duplicates	UNIQUE(email)
490	Index	Data structure for fast search	Enterprise	Fast lookup by email	BTREE index
491	Unique Index	Index ensuring uniqueness	Enterprise	Email uniqueness	unique(email)
492	Composite Key	PK made of multiple columns	Enterprise	orderId+productId	(order_id, product_id)
493	Join	Combine tables based on keys	SQL	Fetch orders with user	JOIN users
494	Inner Join	Only matching	SQL	User with	INNER JOIN

		records		orders	
495	Left Join	All from left + matches	SQL	Users even without orders	LEFT JOIN
496	Right Join	All from right + matches	SQL	rarely used	RIGHT JOIN
497	Full Outer Join	All records from both tables	SQL	reconciliation reports	FULL JOIN
498	Query	SQL command to fetch/modify	SQL	Data extraction	SELECT
499	DDL	Data Definition Language	SQL	Create/alter schema	CREATE TABLE
500	DML	Data Manipulation Language	SQL	Insert/update /delete	INSERT
501	DCL	Data Control Language	SQL	Privileges control	GRANT
502	TCL	Transaction Control Language	SQL	Commit/rollb ack	COMMIT
503	SELECT	Read data	SQL	Fetch records	SELECT *
504	INSERT	Create record	SQL	Add user	INSERT INTO users
505	UPDATE	Modify record	SQL	Update status	UPDATE orders SET
506	DELETE	Remove record	SQL	Delete old logs	DELETE FROM logs
507	WHERE clause	Filtering condition	SQL	User by id/email	WHERE email=?
508	GROUP BY	Aggregation grouping	SQL	Sales by category	GROUP BY category
509	HAVING	Filter grouped results	SQL	Only totals > 100	HAVING SUM()>100
510	ORDER BY	Sort results	SQL	Sort by date desc	ORDER BY created_at DESC

511	LIMIT/OFFSET	Pagination	SQL	Paged API response	LIMIT 10 OFFSET 20
512	Aggregation	Summary functions	SQL	Count, sum	COUNT(), SUM()
513	COUNT()	Counts rows	SQL	total users	COUNT(*)
514	SUM()	Sum numeric values	SQL	total revenue	SUM(amount)
515	AVG()	Average values	SQL	average order	AVG(amount)
516	MIN()/MAX()	Minimum/maximum values	SQL	oldest/latest	MAX(created_at)
517	ACID	Transaction properties	DB	Banking/payment correctness	Atomicity etc
518	Atomicity (DB)	All operations succeed or none	DB	Payment debit+credit	rollback
519	Consistency (DB)	DB moves between valid states	DB	FK constraints valid	rules
520	Isolation (DB)	Transactions don't interfere	DB	Parallel updates safety	isolation levels
521	Durability (DB)	Committed data persists	DB	No data loss after crash	commit log
522	Transaction	Logical unit of work	SQL	Payment workflow	begin→commit
523	Auto-commit	Each SQL commits automatically	JDBC	default mode	conn.setAutoCommit(true)
524	Commit	Save transaction changes	SQL/JDBC	finalize write	conn.commit()
525	Rollback	Undo transaction changes	SQL/JDBC	cancel payment	conn.rollback()
526	Savepoint	Intermediate rollback point	JDBC	partial rollback	conn.setSavepoint()

527	Isolation Level	Defines transaction isolation	JDBC/DB	avoid dirty reads	READ_COMMITTED
528	READ_UNCOMMITTED	Allows dirty reads	DB	rarely used	fastest unsafe
529	READ_COMMITTED	Prevent dirty reads	DB	common default	Oracle default
530	REPEATABLE_READ	Prevent non-repeatable reads	DB	report correctness	MySQL default
531	SERIALIZABLE	Highest isolation	DB	financial transfers	slow but safe
532	Dirty Read	Reading uncommitted data	DB	wrong report	read dirty
533	Non-repeatable Read	Same query returns diff rows	DB	concurrent update issue	read twice differs
534	Phantom Read	New rows appear in re-query	DB	pagination anomalies	phantom rows
535	JDBC	Java Database Connectivity API	1.1+	Java ↔ DB	java.sql.*
536	JDBC Driver	DB vendor driver implementation	1.1+	MySQL driver jar	mysql-connector-j
537	DriverManager	Loads drivers + returns connection	1.1+	Simple JDBC connect	DriverManager.getConnection()
538	DataSource	Connection factory (preferred)	Enterprise	pooling integration	HikariCP DataSource
539	Connection	DB session/connection	JDBC	execute SQL	Connection conn
540	Statement	Executes static SQL	JDBC	simple queries	createStatement()

541	PreparedStatement	Precompiled param query	JDBC	secure+fast	WHERE email=?
542	CallableStatement	Executes stored procedures	JDBC	legacy DB logic	{call proc(?)}
543	ResultSet	Cursor over returned rows	JDBC	iterate query output	rs.next()
544	ResultSet Cursor	Pointer to current row	JDBC	row traversal	next/previous
545	Batch Processing	Group multiple SQL statements	JDBC	bulk insert/update	addBatch()
546	executeBatch()	Executes batch statements	JDBC	faster bulk operations	stmt.executeBatch()
547	Parameter Binding	Setting values in prepared statement	JDBC	avoid SQL injection	ps.setString(1,email)
548	SQL Injection	Attack via SQL input	Security	prevent hacking	use PreparedStatement
549	Connection Pool	Reuse connections for speed	Enterprise	high throughput services	HikariCP
550	HikariCP	High performance JDBC pool	modern	Spring Boot default	hikari
551	Max Pool Size	Max concurrent connections in pool	Pool	tuning performance	maximumPoolSize=20
552	Minimum Idle	Minimum idle connections	Pool	maintain readiness	minimumIdle=5
553	Connection Timeout	Wait time for a connection	Pool	avoid request hang	timeout config
554	Leak Detection	Detect unclosed connections	Pool	prevent DB exhaustion	leakDetectionThreshold
555	ORM	Object Relational Mapping	Enterprise	map Java objects to DB	Hibernate

556	JPA	Java Persistence API spec	Java EE	persistence standard	javax/jakarta.persistence
557	Hibernate	Popular JPA ORM implementation	Enterprise	automatic SQL generation	Hibernate ORM
558	Entity	Persistent object mapped to table	JPA	User entity	@Entity
559	@Entity	Marks class as JPA entity	JPA	ORM mapping	@Entity class User
560	@Table	Specifies DB table name	JPA	rename mapping	@Table(name="users")
561	@Id	Marks primary key field	JPA	unique identifier	@Id Long id
562	@GeneratedValue	Auto-generate PK	JPA	auto increment	GenerationType.IDENTITY
563	GenerationType.IDENTITY	DB auto-increment strategy	JPA	MySQL identity	IDENTITY
564	GenerationType.SEQUENCE	DB sequence strategy	JPA	Oracle/Postgres	SEQUENCE
565	GenerationType.AUTO	Provider chooses strategy	JPA	default strategy	AUTO
566	@Column	Map field to column	JPA	column constraints	@Column(nullable=false)
567	@Transient	Ignore field persistence	JPA	runtime-only fields	calculated field
568	@OneToOne	One-to-one relationship mapping	JPA	user-profile	@OneToOne
569	@OneToMany	One-to-many relationship	JPA	user-orders	list of orders
570	@ManyToOne	Many-to-one mapping	JPA	orders-user	Order -> User
571	@ManyToMan	Many-to-many	JPA	roles-	join table

	y	mapping		permissions	
572	Join Table	Table used for many-to-many mapping	DB/JPA	role_user	join table
573	@JoinColumn	FK join column mapping	JPA	user_id FK	@JoinColumn(name ="user_id")
574	@JoinTable	Specifies join table metadata	JPA	many-to-many	join table config
575	Cascade	Propagate persistence operations	JPA	save user saves orders	cascade = ALL
576	CascadeType. ALL	propagate all operations	JPA	save/update/ delete	ALL
577	CascadeType.P ERSIST	propagate persist only	JPA	save children	PERSIST
578	CascadeType. MERGE	propagate merge	JPA	update children	MERGE
579	CascadeType. REMOVE	propagate delete	JPA	delete dependent	REMOVE
580	FetchType	Lazy/eager loading rule	JPA	performance tuning	LAZY/EAGER
581	FetchType.LA ZY	Load relation only when needed	JPA	avoid heavy queries	default for collections
582	FetchType.EA GER	Load immediately	JPA	small relations	may cause slowness
583	LazyInitializati onException	Access lazy relation outside session	Hibernate	common bug	session closed
584	Persistence Context	JPA first-level cache	JPA	avoid duplicate queries	entity manager cache
585	First Level Cache	Cache within session/entity manager	Hibernate	performance gain	within tx

586	Second Level Cache	Shared cache across sessions	Hibernate	reduce DB hits	Ehcache/Redis
587	Query Cache	Cache query results	Hibernate	expensive analytics queries	cache query
588	N+1 Problem	1 query + N queries for children	Hibernate	performance issue	lazy relations
589	Eager Fetch Problem	Too many joins/data load	Hibernate	slow API response	huge entity graph
590	JPQL	JPA Query Language (entity-based)	JPA	queries by entity fields	SELECT u FROM User u
591	HQL	Hibernate Query Language	Hibernate	like JPQL	HQL queries
592	Criteria API	Typesafe query building	JPA	dynamic search filters	criteria builder
593	Named Query	Predefined query with name	JPA	reuse queries	@NamedQuery
594	Native Query	Raw SQL executed via ORM	JPA	complex DB queries	createNativeQuery()
595	Pagination	Page-based data retrieval	SQL/JPA	UI tables	Pageable
596	Optimistic Locking	Locking using version field	JPA	prevent lost updates	@Version
597	Pessimistic Locking	DB-level lock rows	JPA	financial consistency	PESSIMISTIC_WRITE
598	@Version	Version column for optimistic lock	JPA	concurrent update safety	@Version int v
599	Dirty Checking	Auto detect changed entities	Hibernate	update without explicit SQL	auto update

600	Transaction Manager	Component controlling commit/rollback	Spring/JTA	microservices transactions	@Transactional

---

✅ **PART 5 complete (481–600)**

If you say **"PART 6"**, I will generate:

✅ **PART 6 (601–720) — Java 8 Functional Programming + Streams + Optional + Date/Time (Expert Terms)**

It will include:

- Lambda ecosystem terminology
- Stream operations advanced terms
- Functional interfaces list terms (Predicate, Function, Supplier, Consumer, etc.)
- Optional best-practices terms
- Java Time API (LocalDateTime, ZonedDateTime, Instant, Duration, Period)
- Parallel stream pitfalls & use cases

Now here is:

✅ **PART 6 (Terms 601–720) — Java 8+ Functional Programming + Streams + Optional + Date/Time API (Real-Time Use + Examples)**

This part is extremely important for:

- modern Java coding standards
- Spring Boot service layer transformations
- data pipelines
- analytics/report generation
- clean null-safe APIs

✅ **Java 1000+ Terminology MASTER LIST (PART 6: 601–720)**

#	Term	Meaning (Detailed)	Version/ Era	Real-Time Use Case	Example
601	Functional Programming	Programming with functions & immutability	8+	Cleaner service transformatio	streams

				ns	
602	Lambda Expression	Anonymous function implementation	8+	Event handlers, filters	x -> x+1
603	Functional Interface	Interface with exactly 1 abstract method	8+	Lambda target type	runnable
604	@FunctionalInterface	Annotation enforcing functional interface	8+	Prevent extra abstract methods	@FunctionalInterface
605	SAM	Single Abstract Method	8+	Lambda compatibility	Predicate has SAM
606	Method Reference	Reference to method/constructor	8+	Cleaner lambda	System.out::println
607	Constructor Reference	Reference to constructor	8+	Factory creation	User::new
608	Function Composition	Combining functions into pipeline	8+	transformations	andThen()
609	Higher-Order Function	Function accepting/returning function	8+	reusable stream operations	map(Function)
610	Pure Function	Output depends only on input	FP	predictability	no side effects
611	Side Effect	Function changes external state	FP	avoid in streams	writing DB inside map
612	Immutability	State cannot change after creation	FP/Java	thread-safe DTOs	record/final fields
613	Fluent API	Chain style method calls	modern	stream pipeline	.filter().map()
614	Stream API	Functional operations on	8+	filter/map/grouping	list.stream()

		sequences			
615	Stream	Lazy computation pipeline	8+	process collections	Stream<User>
616	Stream Source	Origin of stream	8+	from List/Set/Array	list.stream()
617	Stream Pipeline	Chain of operations on stream	8+	data flow	source→ops→terminal
618	Intermediate Operation	Lazy transformation returning stream	8+	filter/map/sorted	.filter()
619	Terminal Operation	Ends stream and produces result	8+	collect/reduce/forEach	.collect()
620	Lazy Evaluation	Intermediate ops run only at terminal	8+	performance	no terminal = no exec
621	Short-Circuiting	Stops early when result found	8+	anyMatch/findFirst	.findFirst()
622	filter()	Keep elements matching predicate	8+	active users only	.filter(u->u.active)
623	map()	Transform elements	8+	convert entity to DTO	.map(UserDTO::from)
624	flatMap()	Flatten nested streams	8+	user->roles flatten	.flatMap(List::stream)
625	distinct()	Remove duplicates	8+	unique emails	.distinct()
626	sorted()	Sort elements	8+	order results	.sorted()
627	limit()	Restrict element count	8+	top 10 users	.limit(10)
628	skip()	Skip first N elements	8+	pagination	.skip(20)

629	peek()	Debug each element in stream	8+	logging pipeline	.peek(System.out::println)
630	forEach()	Execute action per element	8+	print/log/publish	.forEach()
631	collect()	Collect to List/Set/Map	8+	build result collection	.collect(toList())
632	reduce()	Reduce into single value	8+	sum totals	.reduce(0,Integer::sum)
633	count()	Count elements	8+	count active users	.count()
634	min()/max()	Find min/max by comparator	8+	highest salary	.max(comp)
635	findFirst()	First element if exists	8+	first match	.findFirst()
636	findAny()	Any element (parallel friendly)	8+	parallel search	.findAny()
637	anyMatch()	True if any satisfies predicate	8+	role check	.anyMatch()
638	allMatch()	True if all satisfy predicate	8+	validate all inputs	.allMatch()
639	noneMatch()	True if none satisfies predicate	8+	ensure no banned user	.noneMatch()
640	Collectors	Utility class for collectors	8+	grouping/joining	Collectors.*
641	Collectors.toList()	Collect results into List	8+	API response list	.toList()
642	Collectors.toSet()	Collect results into Set	8+	unique set	toSet()
643	Collectors.toMap()	Collect results into Map	8+	id→user mapping	toMap(User::id,u->u)

644	groupingBy()	Group elements by key	8+	group users by city	groupingBy(User::city)
645	partitioningBy()	Split into 2 groups by predicate	8+	active/inactive split	partitioningBy(User::active)
646	mapping()	Collector mapping step	8+	extract field lists	mapping(User::name,toList())
647	counting()	Collector to count group size	8+	count users per role	groupingBy(role,counting())
648	summarizingInt()	Stats collector (min,max,sum,avg)	8+	salary analytics	summarizingInt(User::age)
649	joining()	Join strings into one string	8+	CSV export	joining(",")
650	downstream collector	Nested collectors	8+	groupBy + counting	groupingBy(..., counting())
651	Collector Characteristics	IDENTITY_FINISH, CONCURRENT, UNORDERED	8+	performance/ parallel tuning	collector metadata
652	Spliterator	Split iterator for parallel processing	8+	parallel streams internal	.spliterator()
653	Stream.of()	Create stream of values	8+	quick stream	Stream.of(1,2)
654	IntStream/LongStream/DoubleStream	Primitive specialized streams	8+	faster numeric ops	IntStream.range()
655	range()	generate sequential ints	8+	loops replacement	IntStream.range(0,n)
656	rangeClosed()	inclusive range	8+	inclusive indexes	rangeClosed(1,10)
657	boxed()	Convert primitive stream → object stream	8+	collect integers	IntStream...boxed()

658	Parallel Stream	Stream using multiple threads	8+	CPU-bound batch processing	parallelStream()
659	parallelStream()	Create parallel stream from collection	8+	bulk processing	list.parallelStream()
660	Common ForkJoinPool	Default pool used by parallel streams	8+	performance & contention	ForkJoinPool.commonPool
661	Parallel Stream Pitfall	Overhead/wrong for IO tasks	8+	do NOT use for DB/network	avoid
662	Non-deterministic Order	parallel stream may change order	8+	incorrect UI outputs	forEach() unordered
663	forEachOrdered()	Maintain encounter order in parallel	8+	ordered output	.forEachOrdered()
664	Optional	Null-safe container object	8+	avoid NPE	Optional<User>
665	Optional.empty()	Empty optional	8+	no value	Optional.empty()
666	Optional.of()	Wrap non-null value	8+	safe return	Optional.of(x)
667	Optional.ofNullable()	Wrap nullable value	8+	handle null	ofNullable(x)
668	Optional.isPresent()	Check value exists	8+	branching	if(opt.isPresent())
669	Optional.ifPresent()	Run if value exists	8+	log when exists	ifPresent(...)
670	Optional.orElse()	fallback immediate default	8+	default user name	orElse("NA")
671	Optional.orElseGet()	fallback lazy supplier	8+	expensive fallback safe	orElseGet(...)

672	Optional.orElseThrow()	throw if absent	8+	mandatory value	orElseThrow()
673	Optional.map()	transform if present	8+	map entity->dto	opt.map()
674	Optional.flatMap()	nested optional flatten	8+	optional chain	flatMap()
675	Optional.filter()	keep value if condition true	8+	conditional optional	filter()
676	Optional misuse	Don't use Optional in fields/serializable DTOs	best practice	avoid overhead	use null internally
677	Date (java.util.Date)	Legacy date object	1.0+	legacy apps	avoid now
678	Calendar	Legacy date-time API	1.1+	legacy scheduling	Calendar.getInstance()
679	SimpleDateFormat	Legacy date formatter (not thread-safe)	1.1+	bug-prone in multi-thread apps	avoid
680	Java Time API	Modern date-time classes	8+	all modern apps	java.time
681	LocalDate	Date without time zone	8+	DOB, holiday	LocalDate.now()
682	LocalTime	Time without date/zone	8+	store working hours	LocalTime.of(10,0)
683	LocalDateTime	Date+time without zone	8+	DB timestamp local	LocalDateTime.now()
684	ZonedDateTime	Date+time with timezone	8+	global apps	ZonedDateTime.now()
685	OffsetDateTime	Date+time with fixed offset	8+	API timestamps	OffsetDateTime.now()
686	Instant	Machine timestamp	8+	audit logs	Instant.now()

		(UTC)			
687	Duration	Time-based amount in seconds/nanos	8+	timeout configs	Duration.ofSeconds(5)
688	Period	Date-based amount (days/months/years)	8+	subscription periods	Period.ofDays(30)
689	ChronoUnit	Units for date/time operations	8+	calculate differences	ChronoUnit.DAYS.between()
690	DateTimeFormatter	Thread-safe date format	8+	API formatting	DateTimeFormatter.ISO_DATE
691	Parsing	Convert String → Date/number	8+	input validation	parse date
692	Formatting	Convert Date/number → String	8+	output formatting	format timestamp
693	Timezone	Region-based time rules	8+	India/US user apps	Asia/Kolkata
694	ZoneId	Represents timezone ID	8+	global service	ZoneId.of("Asia/Kolkata")
695	ZoneOffset	Fixed UTC offset	8+	offset based timestamps	ZoneOffset.UTC
696	Epoch	Time since 1970-01-01 UTC	8+	timestamp storage	epoch millis
697	epochMilli	milliseconds from epoch	8+	DB store numeric timestamp	Instant.now().toEpochMilli()
698	Temporal	Base interface for time objects	8+	generic time operations	Temporal
699	TemporalAdjuster	Custom date adjustment rule	8+	next working day	custom adjuster
700	Clock	Time source abstraction	8+	testing time-based logic	Clock.fixed()

701	Predicate	Function returning boolean	8+	filtering rules	Predicate<User>
702	Function	Input→output mapping function	8+	DTO mapping	Function<User,User DTO>
703	Consumer	Takes input, no return	8+	logging/send message	Consumer<User>
704	Supplier	Provides values, no input	8+	lazy object creation	Supplier<String>
705	BiPredicate	Predicate with 2 args	8+	validate 2 values	(a,b)->a>b
706	BiFunction	Function with 2 inputs	8+	combine values	(a,b)->a+b
707	BiConsumer	Consumer with 2 inputs	8+	map iteration	(k,v)->log()
708	UnaryOperator	Function same input/output type	8+	string transform	s->s.trim()
709	BinaryOperator	Takes 2 same types returns same type	8+	reduction sum	(a,b)->a+b
710	Streams vs Loops	Streams for declarative style, loops for performance/control	8+	service layer transformations	choose wisely
711	Optional Best Practice	Use Optional as return type not param/field	8+	clean API design	return Optional
712	Side-effect free stream	avoid modifying external state	8+	thread safety	no list.add() in map
713	Stateless lambda	lambda should not depend on mutable external state	8+	safe parallel streams	pure functions

714	Stateful operation	depends on previous state	8+	slow in parallel	sorted()
715	Collector mutability	collector internal container mutability	8+	parallel correctness	concurrent collector
716	Stream reuse	Streams cannot be reused after terminal	8+	avoid IllegalStateException	new stream each time
717	IllegalStateException (stream)	thrown when reusing stream	8+	common bug	stream already operated
718	Functional Style DTO Mapping	mapping with method refs	8+	REST response mapping	.map(UserDTO::new)
719	MapReduce concept	map + reduce aggregation model	FP	analytics computation	sum/count
720	Declarative Programming	What to do not how	8+	clean readable code	stream pipeline

---

✅ **PART 6 complete (601–720)**

If you say **"PART 7"**, I will generate:

✅ **PART 7 (721–840) — Java 9–21 Modern Java Features (Modules, Records, Sealed, Pattern Matching, Virtual Threads)**

It will include:

- module system terms
- JShell terms
- var, text blocks
- switch expressions
- record, sealed classes
- pattern matching for instanceof/switch
- virtual threads + structured concurrency terms

Now here is:

✅ **PART 7 (Terms 721–840) — Java 9 → Java 21 Modern Features (Modules, Records, Sealed, Pattern Matching, Virtual Threads)**

This part is **latest Java terminology** used in:

- modern microservices
- cloud-native deployments
- high concurrency systems
- clean domain-driven design

✅ **Java 1000+ Terminology MASTER LIST (PART 7: 721–840)**

#	Term	Meaning (Detailed)	Version/ Era	Real-Time Use Case	Example
721	Modern Java	Java improvements since 9 (LTS focus)	9+	microservices modernization	Java 17/21
722	LTS	Long Term Support releases	8/11/17/ 21	enterprise production	upgrade planning
723	Jigsaw Project	Java module system initiative	9	modularizatio n	Jigsaw
724	Java Platform Module System (JPMS)	Module system for strong encapsulation	9	modular apps + small runtime	modules
725	Module	Group of packages with boundaries	9	split monolith into modules	module app {}
726	module-info.java	Descriptor file for module	9	declares requires/expo rts	module config
727	requires	Module dependency directive	9	use other modules	requires java.sql;
728	exports	Expose packages to others	9	API boundaries	exports com.api;
729	opens	Allow reflection deep access	9	frameworks like Spring	opens com.model;

730	requires transitive	Re-export module dependency	9	library modules	requires transitive
731	provides ... with	Service provider config	9	plugin architecture	SPI
732	uses	Declare service consumption	9	load provider implementations	uses
733	ServiceLoader	Discover implementations at runtime	6+ used heavily in 9+	plugin systems, JDBC drivers	ServiceLoader.load()
734	Encapsulation (modules)	Restrict internal access strongly	9	prevent misuse	hidden packages
735	IllegalAccessError (modules)	Access denied due to module rules	9	migration issue	reflection blocked
736	--add-opens	JVM arg opening modules for reflection	9+	fix Spring/Hibernate reflection	add-opens
737	--add-exports	Export internal package to module	9+	legacy libs compatibility	add-exports
738	jlink	Build custom runtime image	9+	small docker images	jlink --module-path
739	jmod	Package module as JMOD	9+	modular distribution	.jmod
740	jdeps	Dependency analyzer tool	8+	migration analysis	jdeps app.jar
741	JShell	Interactive REPL for Java	9	quick experiments	jshell
742	REPL	Read-Eval-Print-Loop	9	fast testing	JShell
743	Multi-release JAR	Different class versions per Java release	9	support multi Java versions	META-INF/versions

744	var (LVTI)	Local variable type inference	10	cleaner code	var list = new ArrayList<>();
745	Local Variable Type Inference	Compiler infers type at compile time	10	reduce verbosity	var
746	Not dynamic typing	var still strongly typed	10	avoid confusion	compile-time type
747	Private interface methods	Allow private helper in interface	9	default method reuse	private void helper()
748	Collection Factory Methods	List.of/Set.of/Map.of	9	immutable config collections	List.of("A")
749	Immutable Collections	Collections that cannot change	9	safe constants	Set.of()
750	Compact Strings	Strings stored as byte[] when possible	9	memory optimization	LATIN1
751	G1 as default GC	G1 collector default	9	balanced throughput/latency	-XX:+UseG1GC
752	HTTP Client (incubator)	Modern HTTP client preview	9/10	microservices REST calls	HttpClient
753	Java 11 HttpClient	Standardized HTTP client	11	API calls without external libs	HttpClient.newHttpClient()
754	TLS 1.3 support	Improved encryption protocol support	11+	secure comms	TLS 1.3
755	Single-file source execution	Run Java file without explicit compile	11	scripting	java Hello.java
756	Flight Recorder	Low overhead profiling/events	11+	prod performance	JFR

	(JFR)			diagnostics	
757	JMC	Java Mission Control tool	11+	analyze JFR recordings	JMC
758	ZGC	Low-latency scalable GC	11+ stable later	huge heap low pause	-XX:+UseZGC
759	Shenandoah GC	Low pause GC (RedHat)	12+	latency-critical services	-XX:+UseShenandoahGC
760	Switch Expression	switch returns value	14+	cleaner mapping	var x = switch(a){...};
761	Arrow Labels	case X -> syntax	14+	less boilerplate	case 1 ->
762	yield (switch)	Return from switch block	14+	multi-line cases	yield value;
763	Text Blocks	Multiline string literal	15	JSON/XML templates	"""
764	Sealed Classes	Restrict which classes can extend	17	secure domain models	sealed class
765	permits	Specifies allowed subclasses	17	sealed control	permits A,B
766	non-sealed	Allows further extension	17	relax restriction	non-sealed class
767	Record	Immutable data carrier class	16	DTOs/entities light-weight	record User(...)
768	Record Components	Fields defined in record header	16	standard DTO fields	record User(int id)
769	Canonical Constructor	Auto generated constructor for record	16	validation customization	public User{}
770	Compact Constructor	Constructor without parameter list	16	validate/normalize values	public User{...}

771	Pattern Matching	Smarter type checks and extraction	16+	cleaner business logic	instanceof pattern
772	Pattern Matching for instanceof	Cast + bind in one step	16	simplify code	if(o instanceof User u)
773	Guarded Pattern	Pattern with extra condition	21	advanced matching	case X x when cond ->
774	Pattern Matching for switch	switch with type patterns	21	route by object type	switch(obj)
775	Null in switch patterns	Handling null explicitly	21	avoid NPE	case null ->
776	Record Pattern	Destructure record components	21	nested DTO extraction	case User(var id,var name)
777	Deconstructio n Pattern	Unpack object structure	21	clean nested matching	record pattern
778	Preview Feature	Feature not final (needs enable)	12+	try new syntax	--enable-preview
779	--enable-preview	Enable preview compilation/run time	12+	testing new features	compile/run flags
780	Incubator Feature	API provided for trial (may change)	9+	test emerging APIs	Incubator
781	Deprecation	Mark feature to be removed in future	all	migration planning	@Deprecated
782	Removal	Deprecated feature removed	9+	breaking changes	Java EE removal
783	Java EE Modules removed	javax.xml.bind etc removed from JDK	11	need dependencies	add JAXB
784	Unicode updates	Updated Unicode	9+	correct text processing	emoji support

		versions support			
785	JEP	JDK Enhancement Proposal	9+	track language changes	JEP list
786	var in lambda params	allow var in lambda args	11	annotation for params	(var x) -> x+1
787	Local-Variable Syntax for lambda	using var inside lambda	11	uniform style	lambda var
788	Garbage Collector improvements	GC enhancements in many releases	9+	high perf	ZGC/G1
789	CDS	Class Data Sharing	10+ improved	faster startup	shared archive
790	AppCDS	Application Class-Data Sharing	10+	faster boot microservices	AppCDS
791	AOT compilation	Ahead-of-time compilation (limited JDK)	9+	faster startup	compile ahead
792	GraalVM Native Image	Compile Java into native binary	external ecosystem	serverless fast startup	native executable
793	Foreign Function & Memory API	Interop with native memory safely	19+ preview / 22+ evolves	call C libs safely	FFM API
794	Panama Project	Native interop initiative	19+	native calls	Panama
795	Loom Project	Lightweight concurrency initiative	21	massive concurrency	virtual threads
796	Virtual Thread	Lightweight thread managed by JVM	21	handle 100k+ requests	Thread.startVirtual Thread()

797	Platform Thread	Traditional OS thread	1.0+	CPU-bound heavy tasks	new Thread()
798	Carrier Thread	OS thread that runs virtual threads	21	internal scheduling	vthread carrier
799	Pinning	Virtual thread forced to carrier due to blocking	21	performance pitfall	synchronized blocking
800	Structured Concurrency	Group tasks with shared lifecycle	21	reliable parallel request fanout	StructuredTaskScope
801	StructuredTaskScope	API for structured concurrency	21	parallel service calls	scope.fork()
802	fork() (scope)	Start subtask in scope	21	parallel calls	scope.fork(() -> call())
803	join() (scope)	Wait for all subtasks	21	gather results	scope.join()
804	ShutdownOnFailure	Cancel tasks if one fails	21	fail fast	ShutdownOnFailure
805	Scoped Values	Safer alternative to ThreadLocal	21	request context propagation	ScopedValue
806	ScopedValue.where()	Bind scoped value for execution	21	tracing/security contexts	where(KEY,val)
807	Continuation	Internal JVM mechanism enabling vthreads	Loom	lightweight suspend/resume	runtime feature
808	Executor with virtual threads	Executor that creates virtual threads	21	async processing at scale	Executors.newVirtualThreadPerTaskExecutor()
809	Virtual thread per task	One vthread for each submitted	21	concurrency w/o pool	vthread executor

	executor	task		sizing	
810	Backpressure	Control producer speed for consumers	modern	avoid overload	bounded queue
811	Reactive vs Virtual Threads	Reactive uses non-blocking streams; vthreads allow blocking style	modern	choose model	WebFlux vs vthread
812	Flow API	Reactive Streams interfaces	9	reactive pipelines	Flow.Publisher
813	Flow.Publisher	Produces items in reactive stream	9	messaging/streaming	Publisher
814	Flow.Subscriber	Consumes items	9	reactive consumers	Subscriber
815	Flow.Subscription	Controls demand (backpressure)	9	request N items	Subscription
816	SubmissionPublisher	Simple publisher implementation	9	event distribution	new SubmissionPublisher<>()
817	Compact Number Formatting	improved formatting features	12+	UI/finance display	1K/1M
818	String::strip	Unicode whitespace trimming	11	input cleaning	s.strip()
819	String::isBlank	blank check	11	validation	s.isBlank()
820	String::lines	split into stream of lines	11	text processing	s.lines()
821	Files.readString	Read file as string quickly	11	config read	Files.readString(p)
822	Optional::isEmpty	Opposite of isPresent	11	cleaner optional	opt.isEmpty()

				checks	
823	Nestmates	Classes sharing private access	11	reflection/perf	nest-based access
824	Local-Variable syntax in switch	pattern matching enhancements	21	clean switch	switch pattern
825	Record as transparent carriers	record best use	16+	DTOs	record user
826	Sealed interfaces	sealed feature for interfaces	17	domain restrictions	sealed interface
827	Enhanced NullPointerException	better NPE messages	14+	debugging	exact null cause
828	Helpful NPE	JVM shows variable path	14+	reduce debug time	Cannot invoke... because x is null
829	JFR Event Streaming	Streaming events live	14+	monitoring production systems	event stream
830	Unicode Escapes	special char support	9+	text correctness	\uXXXX
831	Switch exhaustiveness	sealed + switch ensures all cases handled	17+	compile safety	no default needed
832	Pattern dominance	more specific patterns checked first	21	switch correctness	avoid unreachable
833	Unnamed patterns	ignore matched value _	21	cleaner patterns	case User(_, var name)
834	Unnamed variables	ignore assigned variable _	21	less clutter	_ = expr
835	Record equals/hash/toString auto	auto generated methods	16	less boilerplate	record auto methods
836	Deprecate for removal	annotation meaning	9+	migration planning	@Deprecated(forRemoval=true)

		removal planned			
837	Compact Object Headers	memory optimization project	modern	memory footprint	JVM feature
838	Vector API	SIMD operations (incubator)	16+ incubator	fast math/ML	VectorSpecies
839	Virtual threads debugging	diagnostics of vthreads	21	thread dumps show vthreads	Jstack
840	Modern Java Migration	upgrading legacy apps to 17/21	9+	enterprise modernization	remove javax, modules

---

✅ **PART 7 complete (721–840)**

If you say **"PART 8"**, I will generate:

✅ **PART 8 (841–1000+) — JVM Internals + GC Tuning + Monitoring Tools + Security + Packaging + Build**

It will include:

- JVM architecture terms
- GC algorithms (Serial/Parallel/CMS/G1/ZGC/Shenandoah)
- JVM flags (Xms/Xmx, metaspace)
- profiling tools: jcmd, jstat, jmap, jstack, JFR, JMC
- bytecode tools: javap
- security: keystore, truststore, JCA/JCE
- packaging: jar/war, fat-jar, shaded jar
- deployment terms: Docker, Kubernetes readiness/liveness from Java angle

Now here is the most advanced and **most important enterprise section**:

✅ **PART 8 (Terms 841–1000+) — JVM Internals + GC Tuning + Monitoring + Tools + Security + Packaging + Deployment**

This part covers terminology used in:

- production performance tuning
- memory leak debugging

- high CPU investigation
- security & encryption in Java apps
- build + packaging + deployment pipelines

✅ Java 1000+ Terminology MASTER LIST (PART 8: 841–1000+)

#	Term	Meaning (Detailed)	Version/ Era	Real-Time Use Case	Example
841	JVM Internals	Internal architecture of JVM execution	1.0+	performance & debugging	GC + JIT
842	Runtime Data Areas	JVM memory areas for execution	1.0+	memory tuning	heap/stack/metaspace
843	Heap Memory	Stores objects created with new	1.0+	microservice memory sizing	-Xmx
844	Young Generation	Heap region for new objects	1.0+	fast allocation cleanup	Eden/Survivor
845	Eden Space	New objects allocated here	1.0+	allocation heavy apps	request DTOs
846	Survivor Spaces (S0/S1)	Objects surviving young GC moved here	1.0+	aging calculation	copy GC
847	Old Generation	Long-lived objects stored here	1.0+	caches, session objects	old gen growth
848	Tenured	Objects promoted to old gen	1.0+	reduce promotion failures	Tuning
849	Promotion	Moving object from young → old	1.0+	GC tuning	premature promotion
850	Promotion Failure	Old gen full prevents promotion	1.0+	causes Full GC	big pause

851	Metaspace	Class metadata memory in native	8+	classloading tuning	MaxMetaspaceSize
852	Code Cache	Stores compiled native code by JIT	1.0+	performance	code cache full
853	Direct Memory	Off-heap memory (ByteBuffer.allocateDirect)	1.4+	high IO performance	Netty buffers
854	Off-Heap Memory	Memory outside heap	modern	reduce GC pressure	direct buffers
855	Stack Memory	Per-thread method call frames	1.0+	recursion depth	-Xss
856	Native Memory	OS-level memory used by JVM	1.0+	debug native leaks	NMT
857	NMT	Native Memory Tracking	8+	analyze native mem usage	-XX:NativeMemoryTracking=summary
858	TLAB	Thread Local Allocation Buffer	1.0+	faster object allocation	per-thread Eden chunk
859	Escape Analysis	JIT optimization to allocate on stack	6+ improved	reduce heap allocations	scalar replacement
860	Scalar Replacement	Break object into scalars to avoid alloc	6+	performance tuning	no object allocated
861	JVM GC	Garbage Collector algorithms	1.0+	memory management	G1/ZGC
862	Serial GC	Single-threaded GC	1.0+	small apps, dev	-XX:+UseSerialGC
863	Parallel GC	Multi-threaded	1.0+	batch jobs	-XX:+UseParallelGC

		throughput GC			
864	CMS GC	Concurrent Mark Sweep (deprecated/removed)	1.4–14	low pause legacy	UseConcMarkSweepGC
865	G1 GC	Garbage-First GC	7+ default 9+	balanced pause/throughput	-XX:+UseG1GC
866	ZGC	Ultra low pause GC	11+	huge heap services	-XX:+UseZGC
867	Shenandoah GC	Low pause concurrent GC	12+	latency critical apps	-XX:+UseShenandoahGC
868	Epsilon GC	No-op GC (does not collect)	11+	test memory footprint	-XX:+UseEpsilonGC
869	Mark Phase	GC marks reachable objects	all	GC internals	mark live
870	Sweep Phase	GC clears dead objects	all	free memory	sweep
871	Compact Phase	Moves objects to reduce fragmentation	all	reduce fragmentation	compaction
872	Fragmentation	Free memory split into chunks	all	OOM even with free mem	heap fragmentation
873	GC Pause	Time app stopped for GC	all	latency tuning	reduce pause
874	Concurrent GC	GC runs with app threads	modern	low pause services	ZGC/Shenandoah
875	STW	Stop-The-World pause	all	bad latency	Full GC stop
876	Safepoint	JVM point where threads can pause	all	GC, deopt	safepoint reached

877	Safepoint Time	Time to reach safepoint	all	perf analysis	“safepoint delay”
878	GC Logs	Text logs of GC activity	all	memory debugging	-Xlog:gc*
879	Unified Logging	Standard JVM logging system	9+	debug performance	-Xlog
880	-Xms	Initial heap size	1.0+	avoid heap resize	-Xms512m
881	-Xmx	Max heap size	1.0+	prevent OOM	-Xmx2g
882	-Xss	Thread stack size	1.0+	recursion/thread count tuning	-Xss512k
883	MaxMetaspaceSize	Limits metaspace	8+	control native mem usage	-XX:MaxMetaspaceSize=256m
884	Heap Dump	Snapshot of heap objects	1.0+	memory leak debugging	.hprof
885	OOME	OutOfMemoryError	1.0+	memory exhaustion	heap space
886	OOME: Metaspace	class metadata leak	8+	dynamic class generation leak	too many classes
887	OOME: Direct buffer memory	direct/offheap exhausted	1.4+	Netty heavy traffic	allocateDirect
888	Heap Dump on OOM	auto dump heap on crash	1.0+	postmortem debugging	-XX:+HeapDumpOnOutOfMemoryError
889	-XX:HeapDumpPath	file path for heap dump	1.0+	store dump for analysis	-XX:HeapDumpPath=/dumps
890	GC Tuning	Optimize GC algorithm/flags	all	reduce pause latency	tune G1
891	Throughput	total work per	perf	batch	ops/sec

		time		processing	
892	Latency	response time delay	perf	API SLA	p99 latency
893	p95/p99 latency	percentile response times	perf	SLA monitoring	p99 < 200ms
894	Warm-up	JVM optimization after startup	all	real performance measurement	JIT warms
895	JIT Optimization	runtime compilation and optimization	all	speed critical apps	hotspot JIT
896	Tiered Compilation	multiple compilation levels	8+	balance startup + performance	C1/C2
897	C1 Compiler	client compiler fast compile	HotSpot	faster startup	tier 1
898	C2 Compiler	server compiler best optimization	HotSpot	peak performance	tier 4
899	Deoptimizatio n	revert optimized code	HotSpot	dynamic runtime changes	uncommon traps
900	On-Stack Replacement (OSR)	compile loops mid-execution	HotSpot	speed loops	OSR compile
901	Bytecode	JVM instruction set	all	class analysis	javap –c
902	javap	Bytecode disassembler tool	JDK	debug compilation	javap -c X.class
903	JITWatch	Tool to analyze JIT compilation logs	ecosystem	performance tuning	JIT logs
904	JMX	Java Management Extensions	1.5+	monitoring JVM metrics	MBeans

905	MBean	Managed bean exposed via JMX	1.5+	custom metrics	MBeanServer
906	VisualVM	Profiling/monitoring tool	ecosystem	CPU/mem analysis	visualvm
907	jconsole	JMX monitoring GUI tool	JDK	view memory/threads	Jconsole
908	jcmd	JVM diagnostic command tool	7+	thread/heap/GC control	jcmd <pid> GC.run
909	jstat	JVM stats tool	JDK	GC stats monitoring	jstat -gc <pid>
910	jmap	Memory map + heap dump tool	JDK	capture heap dump	jmap -dump:live
911	jstack	Thread dump tool	JDK	detect deadlocks	jstack <pid>
912	Thread Dump	Snapshot of thread states	all	find deadlocks/high CPU	jstack
913	CPU Profiling	Identify CPU-heavy methods	tooling	optimize performance	sampling profiler
914	Allocation Profiling	Find object allocation hotspots	tooling	reduce GC	memory profiling
915	Sampling Profiler	Low overhead periodic sampling	tooling	prod-safe profiling	async-profiler
916	Instrumentation	Bytecode modification at runtime	1.5+	APM agents	javaagent
917	javaagent	JVM arg to attach agent	1.5+	APM monitoring	-javaagent:newrelic.jar
918	APM	Application Performance Monitoring	enterprise	observability	Datadog/NewRelic

919	OpenTelemetry	Standard telemetry API	modern	tracing/metrics/logs	OTel SDK
920	Distributed Tracing	Trace request across microservices	modern	debug latency	traceId/spanId
921	Span	Unit of work in trace	modern	service timing	db call span
922	TraceId	Unique id for request trace	modern	cross-service correlation	128-bit id
923	Logging	writing events with levels	all	production debugging	SLF4J
924	Log Levels	TRACE/DEBUG/INFO/WARN/ERROR	all	severity based logs	INFO
925	SLF4J	Logging facade API	modern	unify log frameworks	LoggerFactory.getLogger()
926	Logback	Logging implementation	modern	Spring Boot default	logback.xml
927	Log4j2	Logging implementation	modern	performance logging	log4j2.xml
928	MDC	Thread context logging	logging	attach traceId to logs	MDC.put("traceId",id)
929	Classpath Hell	dependency conflicts in runtime	build	NoSuchMethodError	version mismatch
930	NoClassDefFoundError	class missing at runtime	all	jar missing	dependency missing
931	ClassNotFoundException	class not found dynamically	all	reflection load fails	Class.forName
932	LinkageError	class version conflict	all	classpath conflict	duplicate classes
933	fat jar	jar with all dependencies	build	deploy microservices	Spring Boot jar
934	shaded jar	fat jar created	build	avoid	maven-shade-plugin

		via shading plugin		dependency conflict	
935	Uber-JAR	Another name for fat jar	build	deployment	big jar
936	WAR	web archive deployed to servlet container	Java EE	legacy deployments	tomcat war
937	EAR	enterprise archive	Java EE	old enterprise apps	app server
938	Servlet Container	web runtime like Tomcat/Jetty	enterprise	host servlets/JSP	Tomcat
939	Application Server	Full Java EE server	Java EE	legacy banking systems	WebLogic
940	Deployment	releasing software to environment	DevOps	prod rollout	jar deploy
941	Docker	Container for packaging runtime	DevOps	microservice containerization	Dockerfile
942	Container Image	OS + runtime + app bundle	DevOps	k8s deployment	docker image
943	Kubernetes (K8s)	Container orchestration	DevOps	scaling microservices	deployments
944	Pod	Smallest k8s deploy unit	DevOps	one service instance	pod
945	Readiness Probe	check app is ready to serve traffic	DevOps	prevent bad routing	/actuator/health
946	Liveness Probe	check app alive else restart	DevOps	auto heal	liveness check
947	Graceful Shutdown	finish requests before stop	enterprise	safe deployments	shutdown hooks
948	Shutdown	code executed	1.3+	cleanup	Runtime.addShutdo

	Hook	on JVM termination		resources	wnHook()
949	Signal Handling	OS signals (SIGTERM etc)	OS/JVM	k8s termination	SIGTERM
950	Rolling Update	deploy new version without downtime	DevOps	zero downtime	rollout strategy
951	Blue-Green Deployment	two envs switch traffic	DevOps	safe release	blue/green
952	Canary Release	small % traffic new version	DevOps	safe testing	canary
953	Config Externalization	keep config outside jar	microservices	env-based config	ENV vars
954	Secrets Management	secure creds (DB password)	security	avoid hardcoding	Vault/K8s secret
955	Java Security	APIs for encryption/signing/auth	1.0+	secure data	JCA/JCE
956	JCA	Java Cryptography Architecture	1.2+	crypto framework	MessageDigest
957	JCE	Java Cryptography Extension	1.2+	advanced encryption	Cipher
958	KeyStore	Storage for private keys/certs	1.2+	SSL configuration	.jks/.p12
959	TrustStore	Stores trusted certificates	1.2+	validate remote SSL cert	Cacerts
960	JKS	Java KeyStore format	legacy	old TLS cert store	.jks
961	PKCS12	Standard key store format	modern	preferred	.p12
962	Certificate	Public key +	security	HTTPS	X.509

		identity			
963	X.509	Standard cert format	security	TLS certificates	X509 cert
964	Cipher	Encryption/decryption component	JCE	encrypt sensitive data	Cipher.getInstance ("AES")
965	AES	Symmetric encryption algorithm	security	data encryption	AES/GCM
966	RSA	Asymmetric crypto algorithm	security	token signing/keys	RSA
967	SHA-256	Hashing algorithm	security	password hashing, signing	MessageDigest
968	MessageDigest	Hash generator API	JCA	checksum	MessageDigest.getInstance("SHA-256")
969	Digital Signature	Validate integrity/authenticity	security	verify payload	Signature API
970	JWT	JSON Web Token	security	stateless auth	bearer token
971	OAuth2	Authorization framework	modern security	login via Google etc	oauth2 flow
972	CSRF	Cross-site request forgery	web security	protect web sessions	csrf token
973	CORS	Cross-origin resource sharing	web security	browser API access	allow origin
974	TLS Handshake	Negotiation for secure connection	HTTPS	secure connection establishment	handshake
975	Cipher Suite	Set of algorithms used in TLS	security	enforce strong crypto	TLS_AES_128_GCM

976	Security Manager	JVM sandbox (deprecated)	legacy	old restriction model	removed direction
977	Code Signing	Verify jar authenticity	security	enterprise distribution	signed jar
978	Jar Signing	Sign jar using jarsigner	JDK	secure distribution	jarsigner
979	jarsigner	Tool to sign jars	JDK	verify integrity	jarsigner tool
980	keytool	Tool to manage keystore certs	JDK	SSL setup	keytool –genkeypair
981	Dependency Management	Manage library versions	build	avoid conflicts	Maven/Gradle
982	Maven	Build + dependency tool	modern	enterprise builds	pom.xml
983	pom.xml	Maven project descriptor	Maven	dependencies/ plugins	<dependency>
984	Gradle	Build automation tool	modern	Android + microservices	build.gradle
985	Dependency Scope	compile/test/runtime/provided	Maven	packaging control	<scope>test</scope>
986	Semantic Versioning	version pattern major.minor.patch	build	upgrade safely	1.2.3
987	BOM	Bill of Materials dependency alignment	build	consistent versions	Spring BOM
988	Shading	Relocate dependencies to avoid conflicts	build	avoid jar hell	shade plugin
989	Reproducible Builds	same input produces same artifact	build	supply chain security	deterministic builds
990	CI/CD	Continuous integration/deployment	DevOps	automated build/deploy	Jenkins

991	Unit Test	Test one unit of code	JUnit	prevent regression	@Test
992	Integration Test	Test DB/API integration	testing	verify full flow	testcontainers
993	Testcontainers	Run Docker DB for tests	modern	integration testing	postgres container
994	Mocking	Simulate dependencies	Mockito	service testing	when(repo.find()).thenReturn()
995	JVM Crash	fatal error causing hs_err log	all	native crash debug	hs_err_pid.log
996	hs_err_pid.log	JVM fatal crash log file	HotSpot	analyze segfault	error log
997	Core Dump	OS dump of crashed process	OS	low-level debug	core file
998	Observability	Ability to understand system behavior	modern	metrics+logs+traces	OTel
999	SRE	Site Reliability Engineering	modern	production reliability	SLIs/SLOs
1000	SLA/SLO/SLI	reliability contracts & targets	modern	uptime/latency goals	99.9%
1001	Memory Leak	objects retained unintentionally	all	OOM issues	growing heap
1002	Heap Retention	references preventing GC	all	leak detection	retained size
1003	GC Pressure	too many allocations causing frequent GC	all	optimize allocations	reduce object creation
1004	Backpressure Strategy	slow down producers when overloaded	modern	stable systems	queue limit

1005	Resilience	system survives failures	microservices	circuit breaker/retry	Resilience4j
1006	Circuit Breaker	stop calling failing service	microservices	prevent cascading failure	open/half-open
1007	Retry Policy	retry failed calls with delay	microservices	network unstable	exponential backoff
1008	Bulkhead Pattern	isolate resources per service	microservices	avoid total collapse	separate pools

---

✅ **PART 8 complete (841–1008)** ✅
 Now we have generated **1000+ Java terms successfully** across all parts!